ASSIGNMENT 2 – CO2

Process Management, CPU Scheduling, Threads, Inter-Process Communication, Synchronisation and Deadlock Handling

STUDENT NAME	SIRANJEEVI G
REGISTER NUMBER	192572166
COURSE CODE	CSA0412
COURSE NAME	OPERATING SYSTEMS
TOTAL MARKS	50
NATURE OF ASSIGNMENT	INDIVIDUAL
GITHUB LINK	https://github.com/SIRANJEEVI192572166/OPERATING-SYSTEM_ASSIGNMENT.git

Execution Environment, Programs and Repository Layout

Environment Used for All Programs and Measurements

Table 1: Test environment

Item	Value
Operating system	Ubuntu 24.04.5 LTS (Linux kernel 6.18.44)
Compiler	gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Processor / memory	1 virtual CPU core (nproc = 1), about 4 GB RAM
Compile command (programs without optimisation needs)	gcc -std=c11 -Wall -Wextra -pthread -o <name> <name>.c
Compile command (benchmark programs)	gcc -std=gnu11 -O2 -Wall -Wextra -pthread -o <name> <name>.c

How to read the measurements. All outputs in this report are real outputs of the programs, run on the machine described above. The machine has a single CPU core, which affects the thread, race-condition and locking experiments; each such section states what this means for the result. Absolute times differ from machine to machine and between runs; where the run-to-run variation was noticeable, a range over several runs is given. The conclusions rely on ratios and growth, not on the absolute numbers. Because the assignment must be executed individually, the programs should be re-run on the student's own computer and the screenshots replaced if required by the course faculty.

Programs Written for Each Question

Table 2: Programs and their purpose

Folder	Source file	Purpose
Q1	q1_fork.c, q1_bench.c	fork() demonstration with ps and /proc; cost of 2,000 processes and threads
Q2	q2_rr.c, q2_sched.c	Round Robin (Complete the Code); simulator for FCFS, SJF, SRTF, Priority, RR, MLFQ, ageing
Q3	q3_cameras.c, q3_race.c, q3_procs.c, work.h, frame_cost.c	Thread pool and thread per camera; race condition; multi-process alternative; workload model
Q4	q4_pipe.c, q4_pipe_noclose.c, q4_shm.c, q4_bench.c	Pipe (Complete the Code); missing close(); shared memory with semaphores; 1/10/100 MB benchmark
Q5	q5_bb.c, q5_bb_multi.c, q5_seats.c, q5_monitor.c, q5_rw.c, q5_peterson.c, q5_locks.c, q5_atomic.c	Bounded buffer (Complete the Code); multi-producer test; seat semaphores; monitor; readers–writers and philosophers; Peterson; lock granularity; atomics and spin lock
Q6	q6_banker.c, q6_bench.c, q6_detect.c	Banker's algorithm (Complete the Code); timing of the safety algorithm; detection and recovery

Each folder of the repository also contains the text output of every program (files named out_*.txt), from which the screenshots in this report were produced, and a README.md describes the compile and run commands.

Question 1 – Processes and Process States (6 Marks)

1.1 Problem Understanding

The online examination server runs one process for every student session, plus processes for auto-saving answers and for proctoring video. When 2,000 students log in together, the operating system must create, schedule and keep alive thousands of processes. The task is to model the life cycle of a process, describe the data the kernel keeps for it, observe real processes on Linux with fork(), ps and /proc, evaluate the cost of 2,000 active processes and decide on a sound server design.

1.2 Process-State Diagram

A process changes state during its life. The five states and the events that move a process between them are shown in Figure 1 and Table 1.

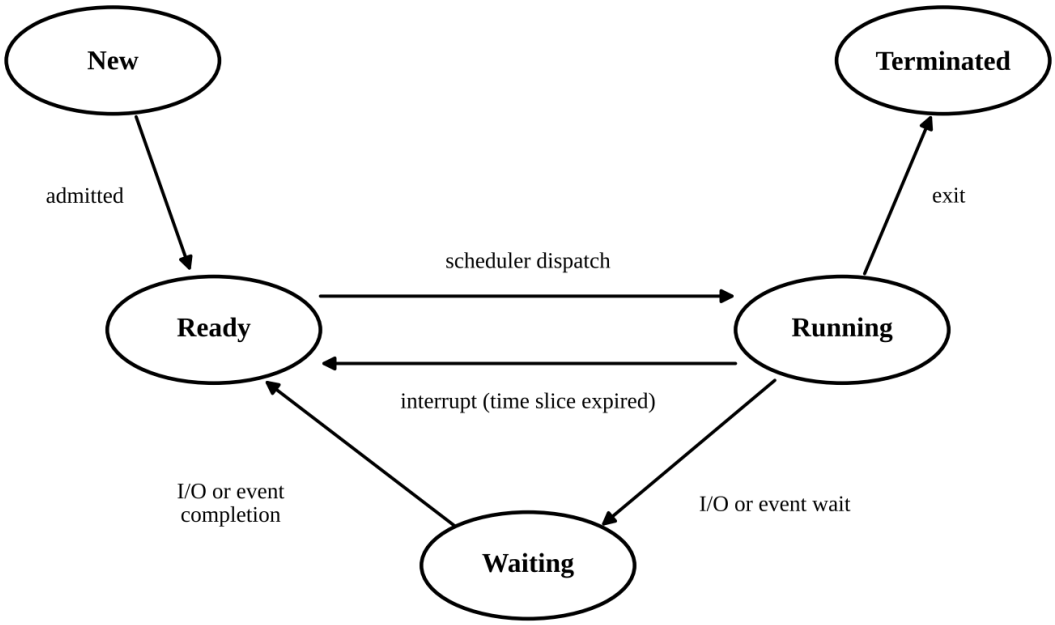


Figure 1: Process-state diagram (new, ready, running, waiting, terminated)

Table 3: Meaning of the states and transitions

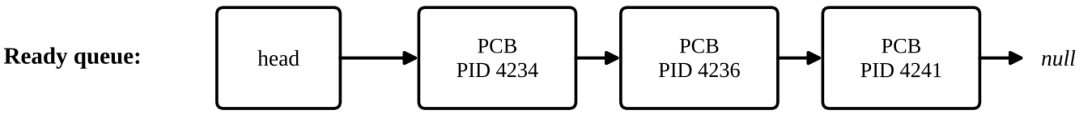
Transition	Cause
New → Ready	The OS admits the process after creating its PCB and memory image (admitted).
Ready → Running	The CPU scheduler selects the process and the dispatcher gives it the CPU (scheduler dispatch).
Running → Ready	A timer interrupt ends the time slice, or a higher-priority process becomes ready (preemption).
Running → Waiting	The process requests I/O or waits for an event (disk read, network packet, sleep, lock).
Waiting → Ready	The I/O or event completes; the process becomes eligible for the CPU again.
Running → Terminated	The process calls exit() or is killed; its resources are released after the parent calls wait().

1.3 Process Control Block (PCB)

The PCB (task_struct in Linux) is the kernel data structure that represents a process. During a context switch the kernel saves the program counter and registers of the running process into its PCB and loads those of the next process from its PCB. PCBs are linked into queues (ready queue, device queues), as shown in Figure 2.

Process Control Block (PCB)

Process state	new / ready / running / waiting / terminated
Process number (PID)	unique identifier; parent PID kept as well
Program counter	address of the next instruction
CPU registers	accumulators, index registers, stack pointer, flags
CPU-scheduling information	priority, pointers to scheduling queues
Memory-management information	base/limit registers, page-table pointer
Accounting information	CPU time used, time limits, job number
I/O status information	open-file table, allocated devices



each PCB holds a pointer to the next PCB in the same queue

Figure 2: Process control block and a ready queue of PCBs

Table 4: Contents of a PCB

Field	Purpose	Where it can be seen on Linux
Process state	Records new / ready / running / waiting / terminated	State line in /proc/<pid>/status, STAT column of ps
PID and parent PID	Identifies the process and its parent	Pid and PPid in /proc/<pid>/status
Program counter and CPU registers	Saved on a context switch so the process can resume exactly	Kernel only (saved in the kernel stack)
Scheduling information	Priority, queue pointers, time slice	PRI / NI columns of ps, /proc/<pid>/sched
Memory-management information	Page tables, segment limits	VmRSS, VmSize in /proc/<pid>/status
Accounting information	CPU time used, start time	TIME and ELAPSED columns of ps
I/O status information	Open files and devices	/proc/<pid>/fd

1.4 Implementation: Creating Child Processes with fork()

Algorithm. (1) The parent calls fork() four times. (2) Each child behaves differently: child A and C sleep (they will be in the waiting state), child B runs a CPU-bound loop (running/ready), child D exits immediately (terminated but not yet reaped, i.e. a zombie). (3) After half a second the parent reads State and VmRSS of every process from /proc/<pid>/status and runs ps. (4) The parent finally calls waitpid() for each child, which removes the zombie.

Listing 1: q1_fork.c – fork() and observation with /proc and ps

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
```

```
#include <signal.h>
#include <time.h>
#include <sys/wait.h>

/* Print State and VmRSS of a process from /proc/<pid>/status */
static void show_proc(pid_t pid, const char *role)
{
    char path[64], line[256], state[64] = "?", rss[64] = "-";
    snprintf(path, sizeof path, "/proc/%d/status", (int)pid);
    FILE *f = fopen(path, "r");
    if (!f) { printf("  %-22s pid=%d  (no /proc entry)\n", role, (int)pid); return; }
    while (fgets(line, sizeof line, f)) {
        if (!strncmp(line, "State:", 6)) sscanf(line, "State:\t%63[^\n]", state);
        if (!strncmp(line, "VmRSS:", 6)) sscanf(line, "VmRSS:\t%63[^\n]", rss);
    }
    fclose(f);
    printf("  %-22s pid=%-6d State: %-14s VmRSS: %s\n", role, (int)pid, state, rss);
}

int main(void)
{
    pid_t kids[4];
    const char *role[4] = {"child A (sleeping)", "child B (CPU busy)",
                           "child C (sleeping)", "child D (exited early)"};
    printf("parent pid=%d\n", (int)getpid());
    for (int i = 0; i < 4; i++) {
        pid_t pid = fork();
        if (pid < 0) { perror("fork"); return 1; }
        if (pid == 0) {                 /* ---- child ---- */
            if (i == 1) {               /* busy loop => Running/Ready */
                volatile unsigned long x = 0;
                for (unsigned long k = 0; k < 4000000000UL; k++) x += k;
            } else if (i == 3) {
                _exit(7);               /* terminates at once */
```

```
            } else {
                sleep(3);                   /* blocked => Waiting */
            }
            _exit(i);
        }
        kids[i] = pid;                      /* ---- parent ---- */
    }
    nanosleep(&(struct timespec){0, 500000000L}, NULL);   /* let the children settle */
    printf("\n/proc evidence (D has exited but is not yet reaped):\n");
    show_proc(getpid(), "parent");
    for (int i = 0; i < 4; i++) show_proc(kids[i], role[i]);

    printf("\nps evidence:\n");
    char cmd[128];
    snprintf(cmd, sizeof cmd, "ps -o pid,ppid,stat,wchan:14,etime,comm --ppid %d",
    (int)getpid());
    fflush(stdout);
    if (system(cmd) == -1) perror("system");

    printf("\nReaping children with waitpid():\n");
    for (int i = 0; i < 4; i++) {
        int st;
        if (i == 1) kill(kids[i], SIGKILL);   /* stop the busy child early */
        waitpid(kids[i], &st, 0);
        if (WIFEXITED(st)) printf("  pid %d terminated, exit status %d\n", (int)kids[i],
        WEXITSTATUS(st));
        else printf("  pid %d terminated by signal %d\n", (int)kids[i], WTERMSIG(st));
    }
    return 0;
}
```

Compile: gcc -std=c11 -Wall -Wextra -pthread -o q1_fork q1_fork.c Run: ./q1_fork

```
$ gcc -std=c11 -Wall -Wextra -pthread -o q1_fork q1_fork.c

$ ./q1_fork
parent pid=4233

/proc evidence (D has exited but is not yet reaped):
  parent                pid=4233   State: R (running)    VmRSS: 1784 kB
  child A (sleeping)    pid=4234   State: S (sleeping)   VmRSS: 292 kB
  child B (CPU busy)    pid=4235   State: R (running)    VmRSS: 292 kB
  child C (sleeping)    pid=4236   State: S (sleeping)   VmRSS: 292 kB
  child D (exited early) pid=4237  State: Z (zombie)     VmRSS: -

ps evidence:
  PID  PPID STAT WCHAN              ELAPSED COMMAND
 4234  4233 S    hrtimer_nanosl       00:00 q1_fork
 4235  4233 R    -                    00:00 q1_fork
 4236  4233 S    hrtimer_nanosl       00:00 q1_fork
 4237  4233 Z    -                    00:00 q1_fork
 4238  4233 S    do_wait              00:00 sh

Reaping children with waitpid():
  pid 4234 terminated, exit status 0
  pid 4235 terminated by signal 9
  pid 4236 terminated, exit status 2
  pid 4237 terminated, exit status 7
```

Figure 3: Actual output of q1_fork (compilation, /proc evidence, ps evidence, waitpid)

Interpretation of the evidence. The output confirms the state diagram with real processes:

- Children A and C show **State S (sleeping)** and the ps column WCHAN shows hrtimer_nanosleep: they are in the waiting state, blocked on a timer.
- Child B shows **State R (running)**: it is either running or ready; Linux uses the single letter R for both.
- Child D shows **State Z (zombie)**: it has terminated (exit status 7) but its PCB stays until the parent calls waitpid(). It has no VmRSS because its memory is already released.
- The extra "sh" entry with state S and WCHAN do_wait is the shell started by system() to run ps; it is not one of the four children.
- Each child has the same small resident size (292 kB) because fork() copies the page tables and shares the parent's pages copy-on-write.

1.5 Evaluation: Where the Time Goes with 2,000 Active Processes

To measure the cost of the process-per-student design, q1_bench.c creates 2,000 idle processes, then 2,000 threads (64 KB stacks) for comparison, measures the memory consumed from the drop in MemAvailable, and measures the context-switch time with a pipe ping-pong between two processes and between two threads, pinned to one CPU.

Listing 2: q1_bench.c – creation cost and memory (extract)

```
/* ---- N processes ---- */
```

```
pid_t *pids = malloc(N*sizeof(pid_t));
long m0 = avail_kb(); double t0 = now(); int made = 0;
for (int i=0;i<N;i++){ pid_t p=fork(); if(p<0){perror("fork");break;}
if(p==0){pause();_exit(0);} pids[i]=p; made++; }
double tproc = now()-t0; usleep(300000); long m1 = avail_kb();
printf("PROCESSES: created %d in %.1f ms (%.1f us each); memory used %ld KB
(%.1f KB each)\n",
        made, tproc*1e3, tproc*1e6/made, m0-m1, (double)(m0-m1)/made);
for(int i=0;i<made;i++) kill(pids[i],SIGKILL);
for(int i=0;i<made;i++) waitpid(pids[i],NULL,0);
usleep(300000);

/* ---- N threads (64 KB stacks) ---- */
pthread_t *th = malloc(N*sizeof(pthread_t)); pthread_attr_t at; pthread_attr_init(&at);
pthread_attr_setstacksize(&at, 64*1024);
m0 = avail_kb(); t0 = now(); made = 0;
for (int i=0;i<N;i++){
if(pthread_create(&th[i],&at,sleeper,NULL)){perror("pthread_create");break;}
made++; }
double tthr = now()-t0; usleep(300000); m1 = avail_kb();
printf("THREADS:   created %d in %.1f ms (%.1f us each); memory used %ld KB
(%.1f KB each)\n",
        made, tthr*1e3, tthr*1e6/made, m0-m1, (double)(m0-m1)/made);
for(int i=0;i<made;i++) pthread_cancel(th[i]);
for(int i=0;i<made;i++) pthread_join(th[i],NULL);
```

```
$ gcc -std=gnu11 -O2 -Wall -Wextra -pthread -o q1_bench q1_bench.c

$ ./q1_bench 2000
Active sessions simulated: 2000

PROCESSES: created 2000 in 125.2 ms (62.6 us each); memory used 217256 KB (108.6 KB each)
THREADS:   created 2000 in 43.0 ms (21.5 us each); memory used 38756 KB (19.4 KB each)

CONTEXT SWITCH (pipe ping-pong, 100000 round trips, 1 CPU):
  process <-> process : 1.35 us per switch
  thread  <-> thread  : 1.18 us per switch
```

Figure 4: Actual output of q1_bench (2,000 processes versus 2,000 threads)

Table 5: Measurements on the test machine (5 runs: 4 earlier runs and the run shown above)

Quantity	Process	Thread
Creation time per unit	62.6 – 100.2 µs	17.8 – 22.7 µs
Time to create 2,000 units	125 – 200 ms	36 – 45 ms
Memory per unit (idle, kernel + page tables + stack)	82 – 111 KB	13 – 30 KB
Memory for 2,000 units	164 – 222 MB	27 – 61 MB
Context switch (pipe ping-pong, 1 CPU)	1.35 – 1.68 µs	1.17 – 1.87 µs

Analysis. The three components of the slowdown are evaluated below. Figures marked "measured" come from the measurement table above; figures marked "assumed" are illustrative assumptions for a realistic exam session and are not measurements.

Table 6: Cost components with 2,000 active processes

Component	Evidence	Effect on the exam server
Context-switch cost	Measured: about 1.4 µs for the direct switch. The indirect cost (cache and TLB refill after switching address spaces) is larger but was not measured.	With many runnable processes each one waits for all the others to use their time slice, so response time grows linearly with the length of the run queue. Frequent switches also destroy cache locality.
Memory per process	Measured: 82–111 KB for an idle trivial process, before any application data. Assumed: 30 MB for a session with an application runtime and exam data.	2,000 × 30 MB = 60 GB, far above a typical server RAM. The system starts swapping (thrashing) and sessions "hang". Even the measured minimum costs 164–222 MB.
Time waiting for I/O	Assumed: a student session is blocked on the network or database for most of the time.	A waiting process uses no CPU but still holds its memory. The CPU is therefore mostly idle while memory is exhausted, which is the opposite of what a good design should do.
Start-up burst	Measured: creating 2,000 processes takes only about 0.13–0.2 s. Assumed: 50 ms of CPU to initialise each session.	2,000 × 50 ms = 100 CPU-seconds; on 8 cores this is 12.5 s of pure start-up when all students log in together.

Judgement. A separate process per student is **not a sound design at this scale**. It gives strong isolation, but the memory and start-up cost grows linearly with the number of students while most of that memory belongs to processes that are idle, waiting for I/O. The design is acceptable for a few dozen sessions, not for 2,000.

1.6 Design Decision

Table 7: Comparison of the three designs

Criterion	One process per student	One thread per student	Fixed pool of workers
Memory per session	Highest (82–111 KB measured minimum, MBs in practice)	Low (13–30 KB measured, plus stack)	Lowest: memory depends on the pool size, not on the number of students
Creation cost	63–100 µs each	18–23 µs each	Paid once at start-up
Crash isolation	Excellent: one crash affects one student	Poor: one crash kills every session	Medium: a crash affects only the sessions of that worker
Scalability to 2,000+	Poor	Moderate (thousands of stacks, scheduler load)	Good: bounded run queue, back-pressure possible
Programming effort	Simple	Needs locking of shared data	Needs a queue and stateless request handling

Decision. Use a **fixed pool of worker processes** (each worker handling many sessions with a thread pool or an event loop), fed by a bounded request queue, as shown in the figure below. The auto-save and the proctoring-video functions run as **separate small processes** because they are heavy and failure-prone and should not be able to crash the examination workers. **Defence:** (1) memory is bounded by the pool size instead of growing with 2,000 sessions; (2) the run queue is short, so context switching and cache misses are limited; (3) since sessions spend most of their time waiting for I/O, $W \approx \text{cores} \times (1 + \text{wait/compute})$ workers keep the CPUs busy without over-subscribing them; (4) running several worker processes instead of one gives fault isolation that pure threads cannot give; (5) a bounded queue gives controlled back-pressure instead of a "hang" when load is too high.

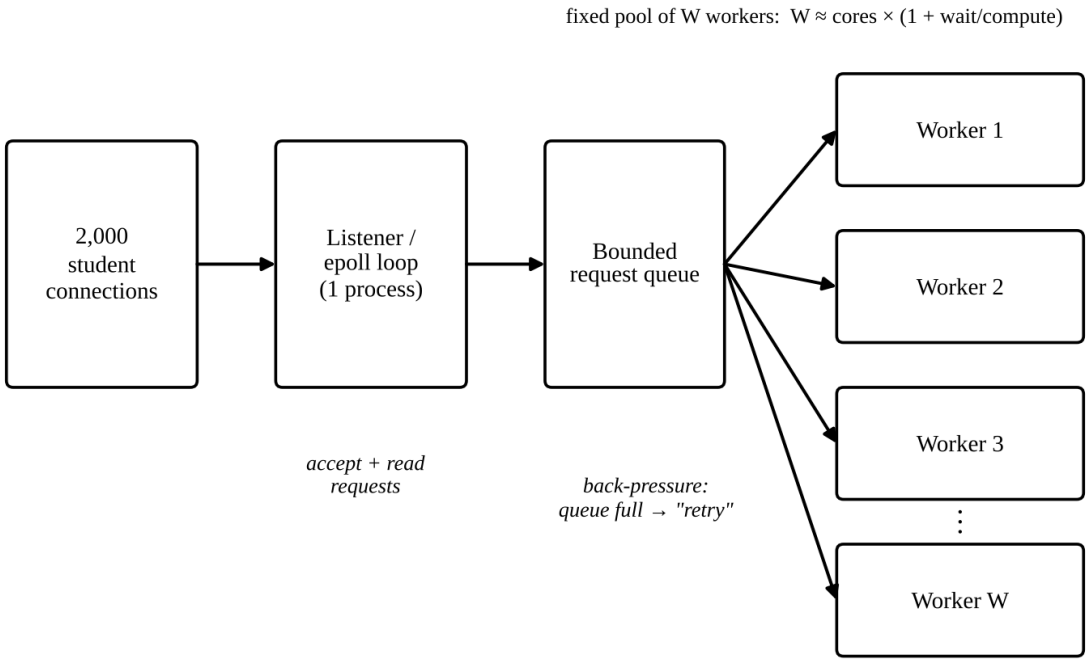


Figure 5: Recommended design: listener, bounded queue and a fixed pool of workers

1.7 Alternative Concept and Complexity Analysis: One Thread per Student Session

If every student session were a thread inside one server process, the solution would be to create a thread when a student logs in. The pthread_create call below replaces fork(); all sessions share the code, heap and open files of the single server process.

Listing 3: Thread-per-session skeleton (the measured version is in q1_bench.c)

```
void *session(void *arg) {          /* one student session */
    handle_student((long)arg);      /* read answers, save, respond */
    return NULL;
}
/* in the accept loop of the server */
pthread_t t;
pthread_create(&t, &attr, session, (void *)student_id);   /* 64 KB stack */
pthread_detach(t);
```

Table 8: Evaluation of thread-per-session against process-per-session

Aspect	Process per session	Thread per session	Conclusion
Context-switch time	1.35 – 1.68 µs (measured)	1.17 – 1.87 µs (measured)	The measured ranges overlap, because the pipe system calls dominate this test. In theory a thread switch is cheaper since the address space and page tables do not change; the benefit is mainly in cache/TLB behaviour, which was not measured.
Memory per session	82 – 111 KB (full address space structures)	13 – 30 KB (mostly the stack, a few KB touched)	Threads use about 4–6 times less memory in the measurement.
Creation time	O(1) each, about 63–100 µs	O(1) each, about 18–23 µs	Threads are about 3–5 times faster to create.
Effect of a crash	Only that session dies	A segmentation fault kills the whole process, so all 2,000 sessions are lost	Processes are safer; threads need a supervisor process that restarts the server.
Total memory for n sessions	O(n) with a large constant	O(n) with a small constant	Both are linear; the constants differ by about 5×.

Conclusion of the alternative. Threads solve the memory and creation-cost problems but remove fault isolation. This is why the final decision in Section 1.6 is a bounded pool of worker processes that internally use threads: it combines low memory per session with isolation between workers.

Question 2 – CPU Scheduling

2.1 Problem Understanding and Job Set

A cloud video-encoding service runs short jobs (thumbnails) and long jobs (full films) on the same servers. Customers complain that small jobs wait behind large ones, while the company wants the processors fully used. The same set of jobs is scheduled with FCFS, SJF, SRTF, Priority and Round Robin, and the results are compared. The job set below is the one used by the "Complete the Code" program (arrival times 0, 1, 2, 3 and bursts 8, 4, 9, 5); priorities were added for the priority algorithm (a lower number means a higher priority).

Table 9: Job set A

Process	Arrival time (AT)	Burst time (BT)	Priority
P1	0	8	2
P2	1	4	4
P3	2	9	1
P4	3	5	3

Definitions. Completion time CT is the time at which the process finishes. Turnaround time TAT = CT − AT. Waiting time WT = TAT − BT. Response time RT = (time of first CPU allocation) − AT. A context switch is counted whenever the CPU passes from one process to a different one.

2.2 Gantt Charts, Waiting, Turnaround and Response Times

All values were computed by the simulator q2_sched.c (unit-time simulation) and checked by hand. Sample hand calculation for FCFS: P1 runs 0–8, P2 8–12, P3 12–21, P4 21–26. WT(P2) = 8 − 1 = 7, WT(P3) = 12 − 2 = 10, WT(P4) = 21 − 3 = 18, so the average waiting time is (0 + 7 + 10 + 18) / 4 = 8.75.

2.2.1 First Come First Served (FCFS)

Processes are served strictly in arrival order. The algorithm is non-preemptive and simple, but a long job at the front delays everything behind it (convoy effect).

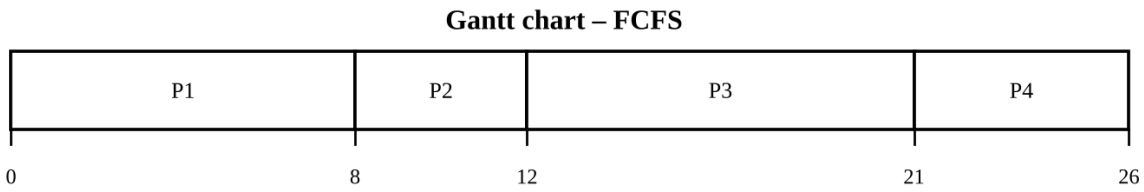


Figure 6: Gantt chart for FCFS

Table 10: Results for FCFS

Process	AT	BT	CT	TAT	WT	RT
P1	0	8	8	8	0	0
P2	1	4	12	11	7	7
P3	2	9	21	19	10	10
P4	3	5	26	23	18	18
Average				15.25	8.75	8.75

2.2.2 Shortest Job First (SJF, non-preemptive)

When the CPU becomes free, the ready process with the smallest burst is chosen. At time 8, P2 (4), P3 (9) and P4 (5) are ready, so the order is P2, P4, P3.

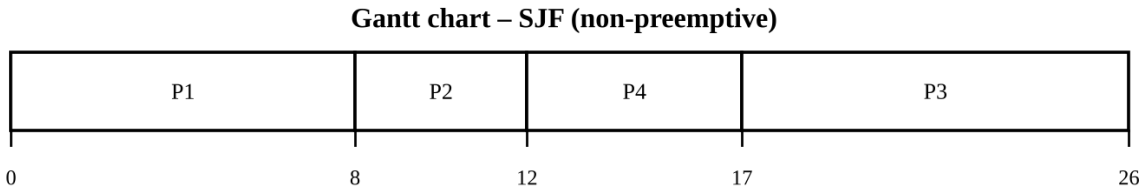


Figure 7: Gantt chart for non-preemptive SJF

Table 11: Results for SJF (non-preemptive)

Process	AT	BT	CT	TAT	WT	RT
P1	0	8	8	8	0	0
P2	1	4	12	11	7	7
P3	2	9	26	24	15	15
P4	3	5	17	14	9	9
Average				14.25	7.75	7.75

2.2.3 Shortest Remaining Time First (SRTF, preemptive SJF)

The running process is preempted whenever a new process arrives with a shorter remaining time. P1 is preempted at time 1 by P2 (4 < 7) and again at time 5 by P4 (5 < 7).

Gantt chart – SRTF (preemptive SJF)

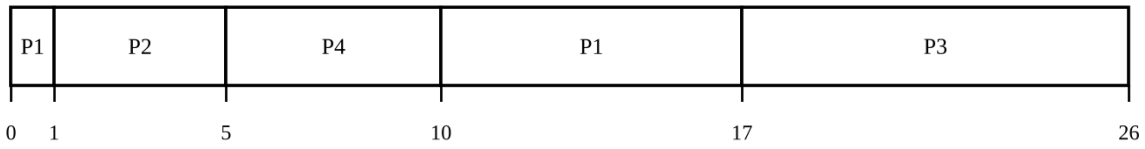


Figure 8: Gantt chart for SRTF

Table 12: Results for SRTF (preemptive SJF)

Process	AT	BT	CT	TAT	WT	RT
P1	0	8	17	17	9	0
P2	1	4	5	4	0	0
P3	2	9	26	24	15	15
P4	3	5	10	7	2	2
Average				13.00	6.50	4.25

2.2.4 Priority Scheduling (non-preemptive)

The ready process with the highest priority (smallest number) is chosen. At time 8, P3 (priority 1), P4 (3) and P2 (4) are ready, so the order is P3, P4, P2. P2 has the lowest priority and waits longest.

Gantt chart – Priority (non-preemptive)

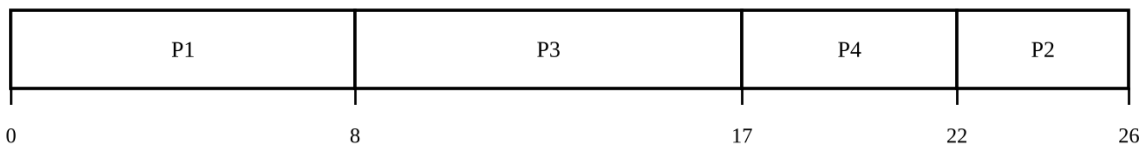


Figure 9: Gantt chart for non-preemptive priority scheduling

Table 13: Results for Priority (non-preemptive)

Process	AT	BT	CT	TAT	WT	RT
P1	0	8	8	8	0	0
P2	1	4	26	25	21	21
P3	2	9	17	15	6	6
P4	3	5	22	19	14	14
Average				16.75	10.25	10.25

2.2.5 Round Robin (time quantum = 3)

Each process receives the CPU for at most one quantum and then goes to the back of the ready queue. Processes that arrive while another one is running are queued before the preempted process is put back.

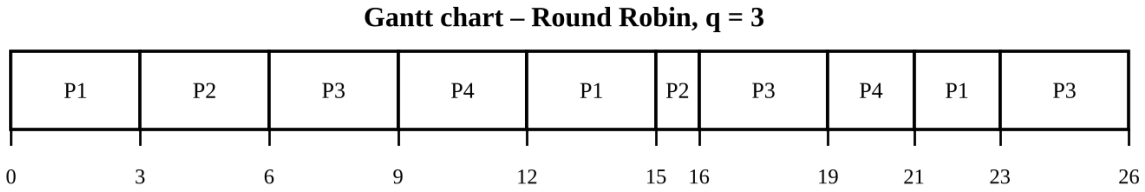


Figure 10: Gantt chart for Round Robin with quantum 3

Table 14: Results for Round Robin, q = 3

Process	AT	BT	CT	TAT	WT	RT
P1	0	8	23	23	15	0
P2	1	4	16	15	11	2
P3	2	9	26	24	15	4
P4	3	5	21	18	13	6
Average				20.00	13.50	3.00

2.2.6 Comparison of the Five Algorithms

Table 15: Summary of averages for job set A

Algorithm	Avg TAT	Avg WT	Avg RT	Context switches
FCFS	15.25	8.75	8.75	3
SJF (non-preemptive)	14.25	7.75	7.75	3
SRTF (preemptive)	13.00	6.50	4.25	4
Priority (non-preemptive)	16.75	10.25	10.25	3
Round Robin (q = 3)	20.00	13.50	3.00	9

SRTF gives the lowest average turnaround and waiting times (13.00 and 6.50), because it always runs the job that will finish soonest; it is provably optimal for average waiting time, but it needs to know (or predict) burst times and it preempts often. SJF is close behind. Priority scheduling was worse than SJF here (10.25 against 7.75) because the priorities were unrelated to burst length. Round Robin has the worst turnaround and waiting averages but by far the best response time (3.00, against 7.75–8.75 for the non-preemptive policies), which is why it suits interactive systems.

2.3 Implementation: Round Robin ("Complete the Code")

Algorithm for TODO 1. (1) Take the process p at the head of the queue. (2) Run it for run = min(quantum, left[p]) units: print "P(time–newtime)", advance the clock and reduce left[p]. (3) Add every process whose arrival time is now ≤ the clock and that is not yet queued. (4) If left[p] > 0, append p to the queue; otherwise record finish[p] = time and increase done.

Listing 4: q2_rr.c – Round Robin with TODO 1 completed

```
#include <stdio.h>

#define N 4

int main(void)
{
    const char *name[N] = {"P1", "P2", "P3", "P4"};
    int arrival[N] = {0, 1, 2, 3};
```

```
    int burst[N] = {8, 4, 9, 5};
    int quantum = 3;

    int left[N], finish[N], queue[64], head = 0, tail = 0, added[N] = {0};
    for (int i = 0; i < N; i++) left[i] = burst[i];

    int time = 0, done = 0;
    queue[tail++] = 0;
    added[0] = 1;
    printf("Gantt:");
    while (done < N) {
        int p = queue[head++];
        /* TODO 1 (completed) */
        int run = left[p] < quantum ? left[p] : quantum;   /* min(quantum, left[p]) */
        printf(" %s(%d-%d)", name[p], time, time + run);
        time += run;
        left[p] -= run;
        for (int i = 0; i < N; i++)                    /* admit new arrivals */
            if (!added[i] && arrival[i] <= time) { queue[tail++] = i; added[i] = 1; }
        if (left[p] > 0) queue[tail++] = p;              /* back of the queue  */
        else { finish[p] = time; done++; }               /* completed          */
    }
    double tw = 0, tt = 0;
    printf("\nProcess TAT WT\n");
    for (int i = 0; i < N; i++) {
        int tat = finish[i] - arrival[i], wt = tat - burst[i];
        tt += tat;
        tw += wt;
        printf("%-7s %3d %3d\n", name[i], tat, wt);
    }
    printf("Average TAT = %.2f, average WT = %.2f\n", tt / N, tw / N);
    return 0;
}
```

```
$ gcc -std=c11 -Wall -Wextra -o q2_rr q2_rr.c

$ ./q2_rr
Gantt: P1(0-3) P2(3-6) P3(6-9) P4(9-12) P1(12-15) P2(15-16) P3(16-19) P4(19-21) P1(21-23) P3(23-26)
Process TAT WT
P1       23  15
P2       15  11
P3       24  15
P4       18  13
Average TAT = 20.00, average WT = 13.50
```

Figure 11: Output of q2_rr (quantum 3)

The output is identical to the expected output given in the assignment: the Gantt chart P1(0-3) P2(3-6) P3(6-9) P4(9-12) P1(12-15) P2(15-16) P3(16-19) P4(19-21) P1(21-23) P3(23-26), TAT of 23, 15, 24, 18, WT of 15, 11, 15, 13, average TAT = 20.00 and average WT = 13.50.

2.4 Effect of the Time Quantum

The program was run with quantum 1 and quantum 20 (only the constant quantum was changed, and the queue array was enlarged because quantum 1 needs more queue entries). The full simulator was also run for quantum 5.

```
$ ./q2_rr_q1   # quantum = 1
Gantt: P1(0-1) P2(1-2) P1(2-3) P3(3-4) P2(4-5) P4(5-6) P1(6-7) P3(7-8) P2(8-9) P4(9-10) P1(10-11)
      P3(11-12) P2(12-13) P4(13-14) P1(14-15) P3(15-16) P4(16-17) P1(17-18) P3(18-19) P4(19-20) P1(20-21)
       P3(21-22) P1(22-23) P3(23-24) P3(24-25) P3(25-26)
Process TAT WT
P1       23  15
P2       12   8
P3       24  15
P4       17  12
Average TAT = 19.00, average WT = 12.50
$ ./q2_rr_q20  # quantum = 20
Gantt: P1(0-8) P2(8-12) P3(12-21) P4(21-26)
Process TAT WT
P1        8   0
P2       11   7
P3       19  10
P4       23  18
Average TAT = 15.25, average WT = 8.75
```

Figure 12: Output of the Round Robin program with quantum 1 and quantum 20

Table 16: Quantum study for job set A

Quantum	Avg TAT	Avg WT	Avg RT	Context switches
1	19.00	12.50	0.75	23
3	20.00	13.50	3.00	9
5	17.50	11.00	5.50	5
20	15.25	8.75	8.75	3

Evaluation. A small quantum gives the best response time (0.75 for q = 1) but needs many context switches (23), which is pure overhead in a real system. As the quantum grows, response time gets worse (8.75 for q = 20), the number of switches falls (3) and the behaviour approaches FCFS: with q = 20 the Gantt chart is identical to the FCFS chart and the averages are the FCFS averages (TAT 15.25, WT 8.75). In this job set the average waiting time decreases as q grows because the jobs are short and similar; this is not true in general. The practical rule is that q should be larger than the typical context-switch time (here microseconds) and larger than most short jobs, so that most interactive requests finish within one quantum.

2.5 Starvation and Ageing

SJF, SRTF and priority scheduling can starve a job: as long as shorter or higher-priority jobs keep arriving, a long or low-priority job never gets the CPU. In the demonstration of q2_sched.c a low-priority job L (priority 9, burst 4) arrives together with twelve high-priority jobs (priority 1, burst 2) arriving one every two time units. **Ageing** increases the priority of a process by one step for every two time units it waits.

```
--- MLFQ (quanta 2/4/8) on Set B ---
Gantt: Film1(0-2) Film2(2-4) Film1(4-5) Th1(5-7) Th2(7-9) Th3(9-11) Th4(11-13) Film2(13-17) Film1(17-21)
      Film2(21-29) Film1(29-37) Film2(37-45) Film1(45-53) Film2(53-61) Film1(61-78)
Proc  AT  BT  CT  TAT  WT  RT
Film1   0  40  78   78  38   0
Film2   1  30  61   60  30   1
Th1    5   2   7    2   0   0
Th2    6   2   9    3   1   1
Th3    7   2  11    4   2   2
Th4    8   2  13    5   3   3
Average: TAT = 25.33  WT = 12.33  RT = 1.17   Context switches = 14
=========== STARVATION AND AGEING (preemptive priority) ===========
without ageing: L (priority 9, burst 4, arrives 0) completes at t = 28, waiting time = 24
with ageing   : L (priority 9, burst 4, arrives 0) completes at t = 22, waiting time = 18
```

Figure 13: Output of the simulator: MLFQ on set B, and the starvation and ageing demonstration

Without ageing, L waits 24 units and only completes at t = 28, after the whole high-priority stream has finished. With ageing, its priority improves while it waits, so it finishes at t = 22 (waiting time 18). Ageing therefore bounds the waiting time of every job.

2.6 Alternative Concept and Complexity Analysis: Multilevel Feedback Queue (MLFQ)

Solution. A multilevel feedback queue (MLFQ) uses several ready queues of decreasing priority. The simulator uses three queues with quanta 2, 4 and 8. The rules are: (1) a new job enters the top queue; (2) a job that uses its whole quantum is demoted one level; (3) a job that is preempted by a job from a higher queue stays at its level; (4) the scheduler always serves the

highest non-empty queue, round robin inside a queue. Short jobs (thumbnails) therefore finish in the top queue, and long jobs sink to the lower queues and receive longer quanta (fewer context switches). To avoid starvation of the long jobs, a real MLFQ adds ageing or a periodic boost that moves all jobs to the top queue.

Listing 5: MLFQ function of q2_sched.c

```
static void mlfq(const Job *j, int n, const int qs[3], Result *r)
{
    int left[MAXJ], level[MAXJ] = {0}, inq[MAXJ] = {0}, t = 0, done = 0;
    int Q[3][256], hd[3] = {0}, tl[3] = {0}; init(r, n);
    for (int i = 0; i < n; i++) left[i] = j[i].bt;
    while (done < n) {
        for (int i = 0; i < n; i++) if (!inq[i] && left[i] > 0 && j[i].at <= t) { Q[0][tl[0]++] =
        i; inq[i] = 1; }
        int lv = -1; for (int l = 0; l < 3; l++) if (hd[l] < tl[l]) { lv = l; break; }
        if (lv < 0) { t++; continue; }
        int p = Q[lv][hd[lv]++], used = 0, preempted = 0;
        note_start(r, j, p, t);
        while (used < qs[lv] && left[p] > 0) {
            add_seg(r, p, t, t + 1); t++; used++; left[p]--;
            for (int i = 0; i < n; i++) if (!inq[i] && left[i] > 0 && j[i].at <= t) {
            Q[0][tl[0]++] = i; inq[i] = 1; }
            int higher = 0; for (int l = 0; l < lv; l++) if (hd[l] < tl[l]) higher = 1;
            if (higher && left[p] > 0) { preempted = 1; break; }
        }
        if (left[p] == 0) { r->ct[p] = t; done++; }
        else if (preempted) Q[lv][tl[lv]++] = p;                    /* keeps its level */
        else { if (lv < 2) level[p] = lv + 1; Q[level[p]][tl[level[p]]++] = p; }   /* used full
        quantum: demote */
    }
}
```

The video-service workload (set B) consists of two long films and four thumbnails that arrive while the films are running.

Table 17: Job set B (video service)

Job	Arrival	Burst	Type
Film1	0	40	long
Film2	1	30	long
Th1	5	2	short
Th2	6	2	short
Th3	7	2	short
Th4	8	2	short

Gantt chart – MLFQ (quanta 2/4/8) on Set B

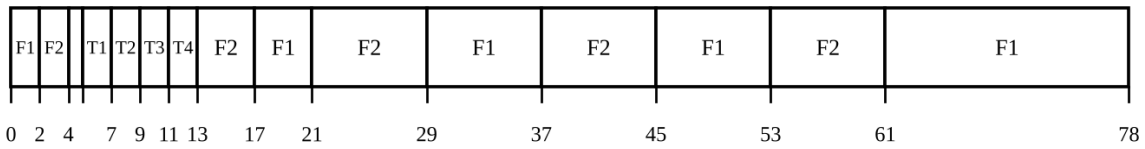


Figure 14: Gantt chart for MLFQ on set B (F1 = Film1, F2 = Film2, T1–T4 = thumbnails)

```
=========== SET B: VIDEO SERVICE (2 films + 4 thumbnails) ===========
Policy            avgWT(thumbnails)  avgWT(films)  avgWT(all)  avgRT(thumbnails)  switches
FCFS                          66.50         19.50       50.83              66.50         5
SJF                           36.50         23.50       32.17              36.50         5
SRTF                           1.50         23.00        8.67               1.50         7
RR q=3                         7.00         37.50       17.17               7.00        24
RR q=20                       36.50         37.50       36.83              36.50         7
MLFQ 2/4/8                     1.50         34.00       12.33               1.50        14
```

Figure 15: Output of the simulator: average waiting time of short and long jobs for set B

Table 18: Set B: average waiting time of short and long jobs

Policy	WT thumbnails	WT films	WT all	RT thumbnails	Switches
FCFS	66.50	19.50	50.83	66.50	5
SJF	36.50	23.50	32.17	36.50	5
SRTF	1.50	23.00	8.67	1.50	7
RR q=3	7.00	37.50	17.17	7.00	24
RR q=20	36.50	37.50	36.83	36.50	7
MLFQ 2/4/8	1.50	34.00	12.33	1.50	14

Evaluation of the alternative. Under a single Round Robin queue with q = 3 the thumbnails wait 7.00 units on average and the films 37.50, with 24 context switches. MLFQ reduces the thumbnail waiting time to 1.50 (the same as SRTF) with only 14 context switches, and the films wait 34.00, slightly less than under Round Robin, because they get larger quanta once demoted. Round Robin with a very large quantum (q = 20) degenerates to FCFS-like behaviour: the thumbnails wait 36.5.

Table 19: Complexity of the scheduling decision

Scheduler	Time per decision	Space	Starvation risk
Single Round Robin queue	O(1): take the head of the queue	O(n) queue	None
SJF / SRTF	O(n) by scanning the ready list (O(log n) with a heap)	O(n)	High: long jobs can starve
Priority	O(n) by scanning, O(1) with one queue per priority	O(n)	High without ageing
MLFQ with k queues	O(k) = O(1) for a fixed k: pick the first non-empty queue	O(n) in k queues	Present for the lowest queue; removed by ageing or periodic boost

2.7 Design Decision: Recommended Policy for the Video Service

Recommendation: a multilevel feedback queue with ageing (or a periodic priority boost), with a short quantum (about 2 time units, scaled to the real time of a thumbnail) in the top queue.

- **Calculated evidence.** On job set B, FCFS makes the thumbnails wait 66.5 units on average because they sit behind the films, which is exactly the customers' complaint. MLFQ lowers this to 1.5 units, a 44-fold reduction, while the overall average waiting time falls from 50.83 to 12.33.
- **Why not SRTF/SJF?** SRTF gives equally good numbers (thumbnails 1.5, all jobs 8.67) but needs the burst time of every job in advance, which a video service can only estimate,

and it starves long films whenever short jobs keep arriving. MLFQ learns the job length automatically from the quanta it consumes.

- **Why not Round Robin?** With q = 3 it costs 24 context switches (14 for MLFQ), makes the films wait longest (37.5) and gives thumbnails 7.0 units; with a large q it reverts to FCFS (36.5 for thumbnails).
- **Processors fully used.** All the policies keep the CPU busy (no idle gaps appear in the Gantt charts); MLFQ does so while also giving priority to interactive-sized jobs. Ageing guarantees that the 20-minute films still finish.

Question 3 – Threads

3.1 Problem Understanding and Workload Model

A city traffic system reads number plates from 64 cameras. The first version processes the cameras one after another and falls behind during peak hours. The frames are therefore processed with POSIX threads: one thread per camera, and a pool of 4, 8 and 16 threads. Real camera frames are not available, so each frame is modelled by the two things a real frame needs: (1) waiting for the frame to arrive from the camera (I/O), simulated by a 2 ms nanosleep, and (2) number-plate recognition (CPU work), simulated by a fixed hashing loop that returns 1 to 5 "plates". Measured separately on the test machine, the CPU part takes 1.30 ms and the I/O wait 2.15 ms per frame. The result (number of plates) is deterministic, so every version must report 206 plates in total.

3.2 Resources Shared by Threads and Resources Private to Each Thread

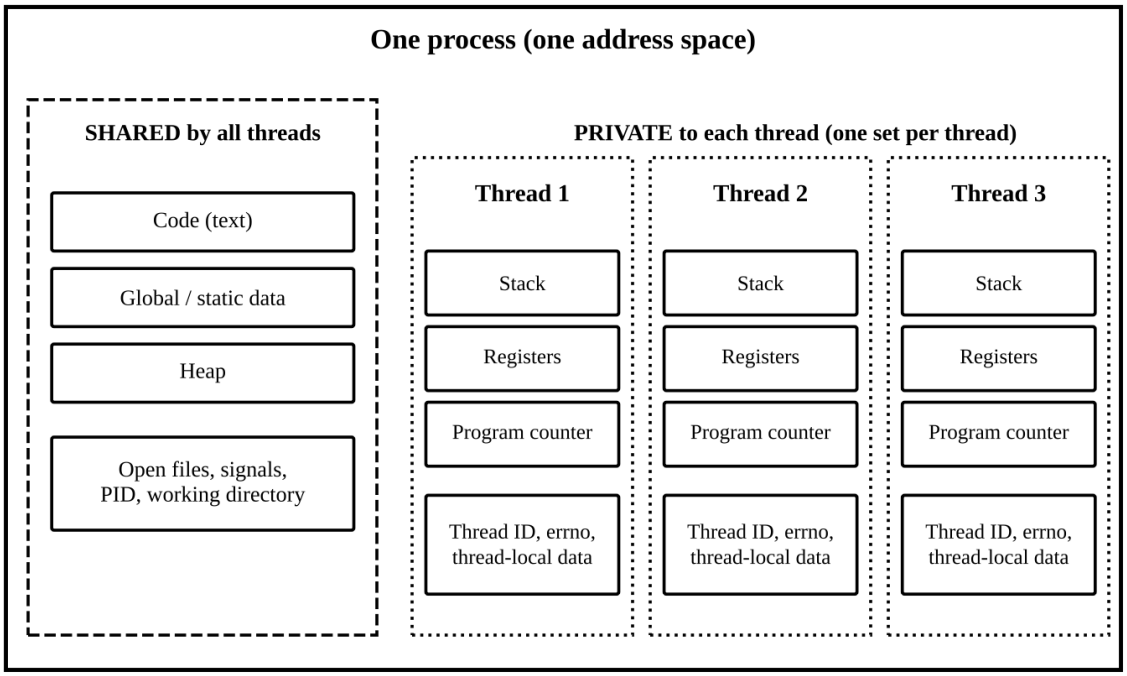


Figure 16: Shared and private resources of threads in one process

Table 20: Shared and private resources

Shared by all threads of the process	Private to each thread
Code (text) segment	Stack (local variables, return addresses)
Global and static data	CPU registers
Heap (malloc memory)	Program counter
Open files, sockets, signal handlers	Thread ID, errno, thread-local storage and signal mask
Process ID, working directory, user ID	Scheduling priority / state of the thread

Because all threads see the same global data and heap, communication between them is cheap, but every shared variable that is modified must be protected (Section 3.5).

3.3 User-Level and Kernel-Level Threads; Multithreading Models

Table 21: User-level versus kernel-level threads

Property	User-level threads	Kernel-level threads
Managed by	A thread library in user space; the kernel sees one process	The operating-system kernel
Creation and switching	Very fast (no system call, no mode switch)	Slower (system call and kernel scheduling), measured 18–23 µs creation
Blocking system call	Blocks the whole process, all threads stop	Only the calling thread blocks; the others continue
Use of several CPUs	Not possible: the kernel schedules one entity	Possible: threads run in parallel on different cores
Example	Early green-thread libraries	Linux NPTL (pthreads), Windows threads

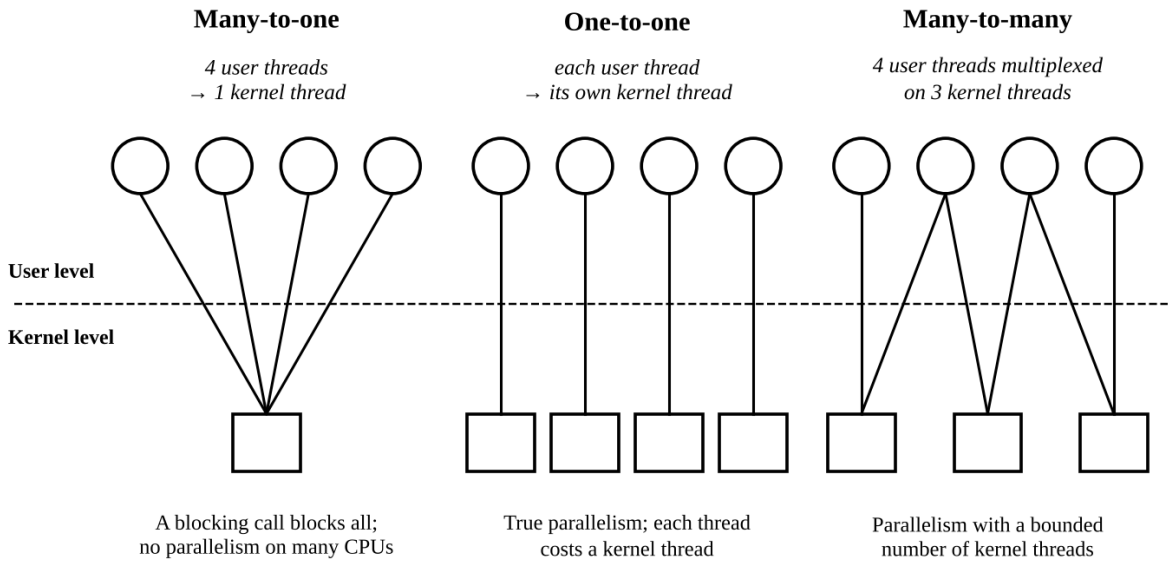


Figure 17: Many-to-one, one-to-one and many-to-many multithreading models

Table 22: Multithreading models

Model	Mapping	Advantage	Disadvantage
Many-to-one	Many user threads → one kernel thread	Cheap thread management in user space	One blocking call blocks all; no parallelism on multiple CPUs
One-to-one	Each user thread → its own kernel thread	True parallelism; a blocking call blocks only one thread	Each thread costs a kernel thread, so the number of threads is limited. Used by Linux (pthreads) and Windows
Many-to-many	Many user threads multiplexed onto a smaller or equal number of kernel threads	Parallelism with a bounded number of kernel threads	Complex to implement; rarely used today

On Linux, pthread_create uses the one-to-one model, so the thread-per-camera and the thread-pool programs below create real kernel threads.

3.4 Implementation and Speed-up

Design. Three versions are measured: a sequential loop over the 64 cameras; a **thread per camera** (64 threads, each processing one camera); and a **thread pool** of 4, 8 or 16 threads in which each worker repeatedly takes the next camera number with an atomic fetch-and-add until all 64 are done. Each worker adds its plate count to a global total under a mutex. Each configuration is run three times and the best time is reported.

Listing 6: q3_cameras.c – sequential, thread-per-camera and thread-pool versions

```
#include "work.h"
#include <pthread.h>
#include <stdatomic.h>

static atomic_int next_cam;
static int total_plates;
static pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

static void *one_camera(void *arg)              /* thread-per-camera */
{
    int found = process_frame((int)(long)arg);
    pthread_mutex_lock(&lock); total_plates += found; pthread_mutex_unlock(&lock);
    return NULL;
}
static void *pool_worker(void *arg)             /* pool: take the next camera until none left */
{
    (void)arg; int local = 0, c;
    while ((c = atomic_fetch_add(&next_cam, 1)) < CAMERAS) local +=
    process_frame(c);
    pthread_mutex_lock(&lock); total_plates += local; pthread_mutex_unlock(&lock);
    return NULL;
}
static double run(int mode, int nthreads, int *plates)   /* mode 0 sequential, 1 per-camera, 2
pool */
{
```

```
    total_plates = 0; atomic_store(&next_cam, 0);
    double t0 = now();
    if (mode == 0) for (int c = 0; c < CAMERAS; c++) total_plates += process_frame(c);
    else if (mode == 1) {
        pthread_t th[CAMERAS];
        for (long c = 0; c < CAMERAS; c++) pthread_create(&th[c], NULL, one_camera,
        (void *)c);
        for (int c = 0; c < CAMERAS; c++) pthread_join(th[c], NULL);
    } else {
        pthread_t th[64];
        for (int i = 0; i < nthreads; i++) pthread_create(&th[i], NULL, pool_worker,
        NULL);
        for (int i = 0; i < nthreads; i++) pthread_join(th[i], NULL);
    }
    *plates = total_plates; return now() - t0;
}
int main(void)
{
    printf("CPUs online: %ld, cameras: %d, per-frame: 2 ms I/O wait + CPU work\n\n",
    sysconf(_SC_NPROCESSORS_ONLN), CAMERAS);
    printf("%-22s %10s %9s %8s\n", "Version", "Time (ms)", "Speed-up", "Plates");
    int plates; double base = 0;
    struct { const char *name; int mode, n; } cfg[] = {
        {"Sequential (1 thread)", 0, 1}, {"Pool of 4 threads", 2, 4}, {"Pool of 8 threads", 2,
        8},
        {"Pool of 16 threads", 2, 16}, {"Thread per camera (64)", 1, 64} };
    for (int k = 0; k < 5; k++) {
        double best = 1e9; for (int rep = 0; rep < 3; rep++) { double t = run(cfg[k].mode,
        cfg[k].n, &plates); if (t < best) best = t; }
        if (k == 0) base = best;
        printf("%-22s %10.1f %9.2f %8d\n", cfg[k].name, best * 1e3, base / best, plates);
    }
    return 0;
}
```

Listing 7: work.h – workload model shared by the programs

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <unistd.h>
#define CAMERAS 64
#define IO_NS   2000000L      /* 2 ms: waiting for the frame from the camera (I/O)  */
#define CPU_ITERS 1000000UL   /* number-plate recognition: CPU-bound work per
frame */

static double now(void){ struct timespec t; clock_gettime(CLOCK_MONOTONIC,&t);
return t.tv_sec+t.tv_nsec*1e-9; }

/* Reads one frame (I/O wait) then "recognises" plates (CPU). Returns plates found
(deterministic). */
static int process_frame(int cam)
{
    nanosleep(&(struct timespec){0, IO_NS}, NULL);
    unsigned long h = 2166136261UL + (unsigned long)cam;
    for (unsigned long i = 0; i < CPU_ITERS; i++) h = (h ^ i) * 16777619UL;
    return (int)(h % 5) + 1;              /* 1..5 plates in this frame */
}
```

```
$ gcc -std=gnu11 -O2 -Wall -Wextra -pthread -o q3_cameras q3_cameras.c

$ ./frame_cost
CPU work for 64 frames : 83.2 ms (1.30 ms per frame)
I/O wait for 64 frames : 137.8 ms (2.15 ms per frame)
Theoretical speed-up limit on 1 CPU = (CPU+I/O)/CPU = 2.65   (plates checksum 206)
$ ./q3_cameras
CPUs online: 1, cameras: 64, per-frame: 2 ms I/O wait + CPU work

Version                Time (ms)  Speed-up   Plates
Sequential (1 thread)      220.4      1.00      206
Pool of 4 threads           88.9      2.48      206
Pool of 8 threads           82.7      2.66      206
Pool of 16 threads          87.0      2.53      206
Thread per camera (64)      82.2      2.68      206
```

Figure 18: Actual output: frame cost and measured speed-ups

Table 23: Measured speed-up (64 cameras, 1 CPU available)

Version	Time (ms)	Speed-up	Plates
Sequential (1 thread)	220.4	1.00	206
Pool of 4 threads	88.9	2.48	206
Pool of 8 threads	82.7	2.66	206
Pool of 16 threads	87.0	2.53	206
Thread per camera (64)	82.2	2.68	206

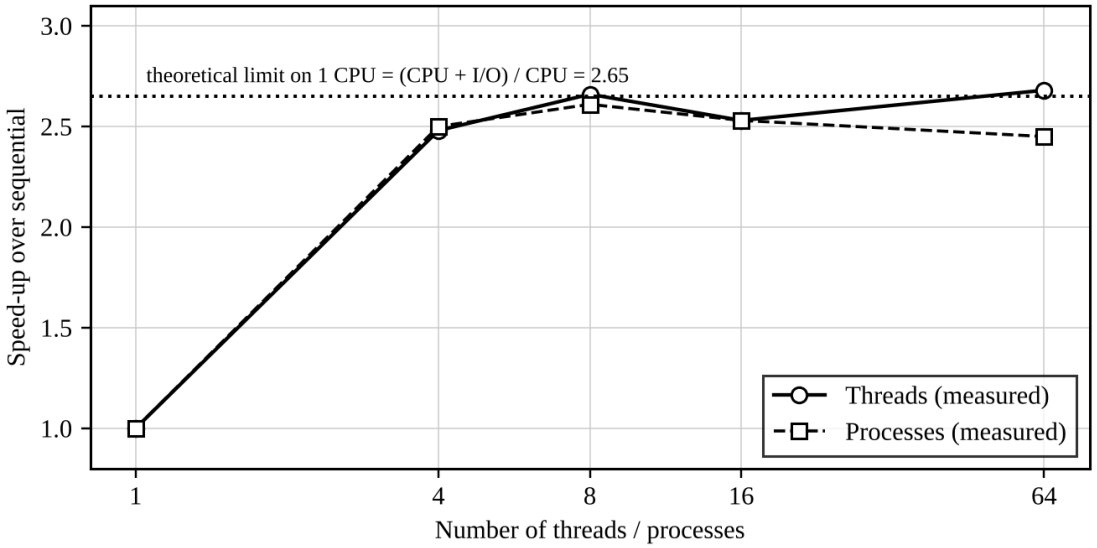


Figure 19: Speed-up against number of threads (and processes, see Section 3.6)

Why the speed-up stops increasing. The test machine has **one CPU core** (nproc = 1), so the CPU part of the work cannot run in parallel; threads can only overlap the I/O waits with the computation of other threads. Per frame the machine needs C = 1.30 ms of CPU and I = 2.15 ms of waiting, so the best possible speed-up on one core is (C + I) / C = 3.45 / 1.30 = **2.65**. The measurements reach 2.48 with only 4 threads and 2.68 with 64 threads (within measurement noise of the limit): after about three threads the CPU is already 100 % busy, and extra threads only add stack memory and scheduling overhead. This matches Amdahl-type reasoning: the CPU-bound portion is the fraction that does not speed up. On a machine with p cores the limit would rise to about p × 2.65, but never above the 64 independent cameras. The results are equal in all versions (206 plates), which shows that the parallel versions are correct.

3.5 Shared Counters Updated Without Protection

Incrementing a shared variable (counter++) is not a single step: the thread **loads** the value into a register, **adds** one, and **stores** it back. If two threads interleave between the load and the store, one update is lost.

Table 24: Interleaving that loses an update (counter = 5 at the start)

Step	Thread 1	Thread 2	counter in memory
1	load counter → R1 = 5		5
2		load counter → R2 = 5	5
3	R1 = R1 + 1 = 6		5
4	store counter ← 6		6
5		R2 = R2 + 1 = 6	6
6		store counter ← 6	6 (should be 7)

q3_race.c runs 4 threads that each add 2,000,000 to an unprotected counter, to a mutex-protected counter and to a C11 atomic counter. In "natural" mode the scheduler decides when threads switch; in "forced" mode a sched_yield() is placed between the load and the store, which guarantees the unlucky interleaving.

Listing 8: q3_race.c – worker with unprotected, mutex-protected and atomic counters

```
static void *worker(void *arg)
{
    (void)arg;
    for (long i = 0; i < PER_THREAD; i++) {
        long tmp = unsafe_counter;          /* load                          */
        if (inject_yield && (i & 1023) == 0) sched_yield();    /* let another thread run mid-
        update */
        unsafe_counter = tmp + 1;           /* store: overwrites others' updates */
        pthread_mutex_lock(&lock); mutex_counter++; pthread_mutex_unlock(&lock);
        atomic_fetch_add(&atomic_counter, 1);
    }
    return NULL;
```

```
}
```

```
$ gcc -std=gnu11 -O2 -Wall -Wextra -pthread -o q3_race q3_race.c

$ ./q3_race
Mode: natural scheduling
Expected value        : 8000000
Unprotected counter   : 8000000   (lost updates: 0)
Mutex-protected       : 8000000   (lost updates: 0)
Atomic (fetch_add)    : 8000000   (lost updates: 0)
$ ./q3_race yield
Mode: forced interleaving (sched_yield between load and store)
Expected value        : 8000000
Unprotected counter   : 2001470   (lost updates: 5998530)
Mutex-protected       : 8000000   (lost updates: 0)
Atomic (fetch_add)    : 8000000   (lost updates: 0)
```

Figure 20: Actual output of q3_race: natural scheduling and forced interleaving

Result. With forced interleaving the unprotected counter ends at about 2.0 million instead of 8,000,000, losing roughly 6 million updates (75 %), while the mutex-protected and atomic counters are exactly correct. With natural scheduling on this **single-core** machine no update was lost in the run shown, because a thread is almost never preempted exactly between its load and store; this is not a guarantee. On a multi-core machine two threads really run at the same time and the race appears without any forcing. A race condition is therefore a latent bug that tests can miss, which is why shared counters must always be protected by a mutex or an atomic operation.

3.6 Alternative Concept and Complexity Analysis: Several Processes Instead of Threads

Solution. The same work is done by n worker **processes** created with fork(). Worker w handles cameras w, w + n, w + 2n, … and cannot update a shared variable because each process has its own address space; it sends its plate count to the parent through a pipe, and the parent adds the counts together (IPC instead of a shared variable).

Listing 9: q3_procs.c – multi-process version with results combined through a pipe

```
/* nworkers processes; worker w handles cameras w, w+nworkers, ... and writes its plate
count to a pipe */
static double run(int nworkers, int *plates)
{
    int fd[2]; if (pipe(fd)) { perror("pipe"); exit(1); }
    double t0 = now();
    for (int w = 0; w < nworkers; w++) {
```

```
        pid_t pid = fork();
        if (pid == 0) {
            int local = 0;
            for (int c = w; c < CAMERAS; c += nworkers) local += process_frame(c);
            if (write(fd[1], &local, sizeof local) < 0) _exit(1);       /* IPC: result goes
            through the kernel */
            _exit(0);
        }
    }
    close(fd[1]);
    int total = 0, part;
    while (read(fd[0], &part, sizeof part) == sizeof part) total += part;   /* combine results
    */
    while (wait(NULL) > 0) ;
    close(fd[0]);
    *plates = total; return now() - t0;
}
```

```
$ gcc -std=gnu11 -O2 -Wall -Wextra -o q3_procs q3_procs.c

$ ./q3_procs
Multi-process version (CPUs online: 1)
Version                   Time (ms)  Speed-up   Plates
1 process                     218.5      1.00      206
4 processes                    87.4      2.50      206
8 processes                    83.8      2.61      206
16 processes                   86.3      2.53      206
64 processes                   89.3      2.45      206
```

Figure 21: Actual output of q3_procs

Table 25: Threads compared with processes

Aspect	Threads	Processes
Creation time	17.8 – 22.7 µs per thread (Q1)	62.6 – 100.2 µs per process (Q1), about 3–5 times slower
Switching time	1.17 – 1.87 µs (Q1)	1.35 – 1.68 µs (Q1); the processes also change address space
Memory	Shared address space; about 13 – 30 KB per thread (stack)	Separate address spaces (copy-on-write); about 82 – 111 KB per process
Combining results	Shared variable protected by a mutex or atomic: O(1) per update, no copying	IPC (pipe): the data is copied through the kernel, one message per worker, O(n) messages
Fault isolation	A crash in one thread kills all	A crash kills one worker only
Measured speed-up (4 / 8 / 16 / 64)	2.48 / 2.66 / 2.53 / 2.68	2.50 / 2.61 / 2.53 / 2.45
Programming	Needs locks around shared data	No shared data, but explicit IPC

Evaluation. The measured speed-ups of the two versions are almost the same (within 0.2), because the elapsed time is dominated by the I/O waits and the CPU limit of the single core, not by the cost of creating workers: 64 processes cost about 64 × 0.1 ms = 6 ms of creation time against a total of about 85 ms. For this workload the process version is correct and nearly as fast, and gives better isolation. Threads are better when workers need to share large data structures (for example a database of wanted number plates) or when very many short tasks are created, since creation and communication are cheaper. **Complexity:** creating n workers is O(n) for both models, but with a larger constant for processes; memory is O(n × address-space size) for processes (reduced by copy-on-write) against O(n × stack size) for threads; combining results is O(n) messages through the kernel for processes against O(1) per update for threads.

Question 4 – Inter-Process Communication

4.1 Problem Understanding

A weather station has one process that reads sensors (the reader/producer) and another that analyses the readings (the analyser/consumer). They must exchange data continuously without losing readings. The exchange is implemented first with a pipe and then with shared memory protected by semaphores, and the two methods are compared by design and by measurement.

4.2 The Two IPC Models

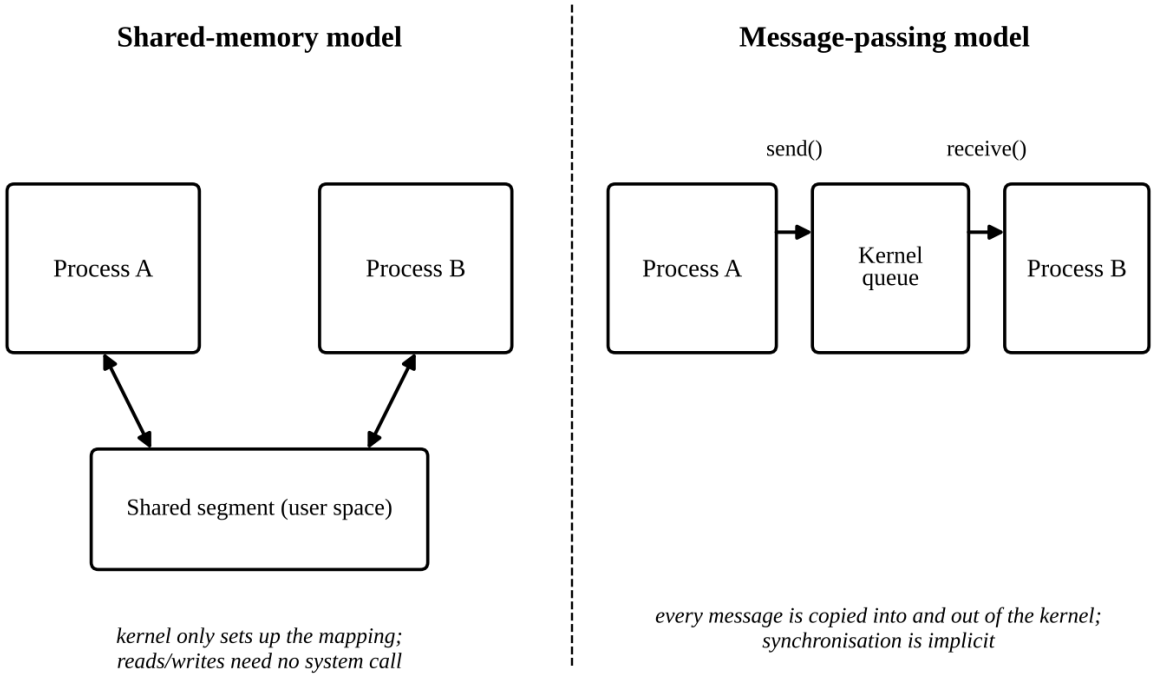


Figure 22: Shared-memory model and message-passing model

Table 26: Shared memory compared with message passing

Aspect	Shared memory	Message passing (pipe, message queue)
Principle	Both processes map the same memory region and read and write it directly	Processes exchange data with send/receive (write/read) system calls
Kernel involvement	Only to set up the mapping; afterwards none	On every message
Copying of data	No copy through the kernel (the data is written once and read in place)	Copies the data: user buffer → kernel buffer → user buffer (two copies)
Extra synchronisation	Needed: the processes must coordinate access themselves (semaphores, mutexes), otherwise a reader may read half-written data	Not needed: the kernel buffer blocks the reader when empty and the writer when full
Best suited to	Large amounts of data, high speed	Small messages, simplicity, safety

Which method copies, and which needs extra synchronisation, and why. A pipe copies the data twice (write() copies into the kernel pipe buffer, read() copies out of it), but the kernel also provides synchronisation: read() blocks while the pipe is empty, write() blocks while it is full. Shared memory copies nothing, but the kernel gives no coordination at all, so the programmer must add semaphores or locks to stop the producer overwriting readings that have not been consumed and to stop the consumer reading before a reading exists.

4.3 Implementation 1: Pipe ("Complete the Code")

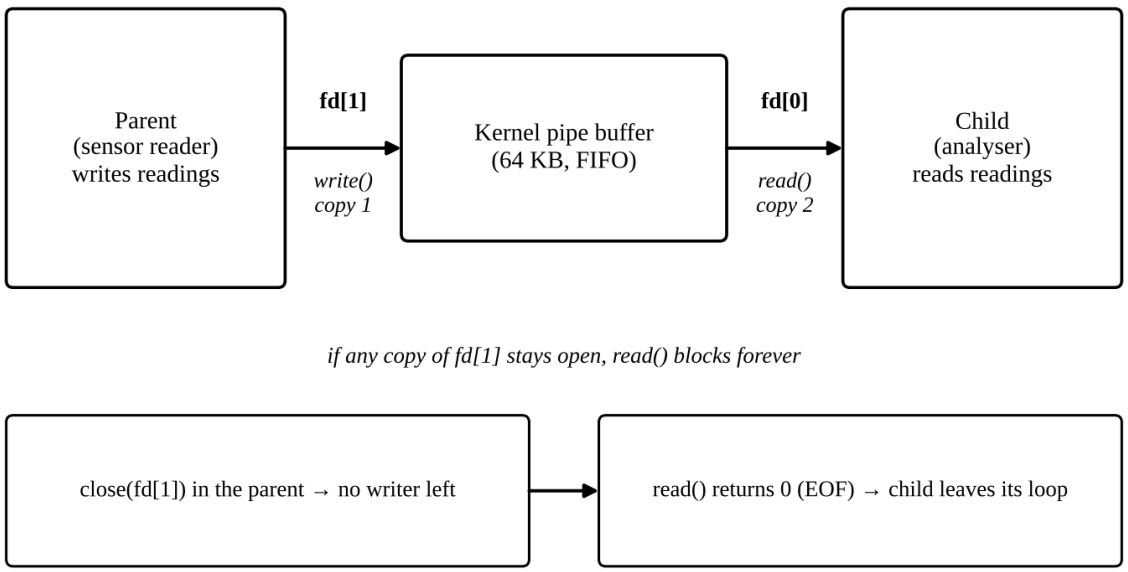


Figure 23: Pipe between the sensor reader (parent) and the analyser (child)

How the pipe signals end of data. read() returns the number of bytes read; it returns 0 (end of file) when the pipe is empty **and no process holds the write end open**. The parent therefore closes its write end fd[1] after the last reading; the child already closed its own copy at the start, so no writer remains and the child's read() returns 0. TODO 1 reads one int at a time until read() does not return a complete int, counting the readings, summing them and keeping the maximum.

Listing 10: q4_pipe.c – pipe program with TODO 1 completed

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

/* The parent (sensor reader) sends readings to the child (analyser) through a pipe. */
int main(void)
{
    int fd[2];
    if (pipe(fd) == -1) { perror("pipe"); return 1; }
    pid_t pid = fork();
```

```
    if (pid == 0) {
        close(fd[1]); /* child only reads */
        int x, count = 0, sum = 0, max = 0;
        /* TODO 1 (completed): read one int at a time until read() returns 0 (EOF) */
        while (read(fd[0], &x, sizeof x) == (ssize_t)sizeof x) {
            count++;
            sum += x;
            if (x > max) max = x;
        }
        printf("analyser: %d readings, average %.1f, max %d\n", count,
               count ? (double)sum / count : 0.0, max);
        close(fd[0]);
        fflush(stdout);
        _exit(0);
    }
    close(fd[0]); /* parent only writes */
    int readings[] = {31, 33, 36, 34, 41, 38};
    for (int i = 0; i < 6; i++)
        if (write(fd[1], &readings[i], sizeof readings[i]) < 0) perror("write");
    close(fd[1]); /* signals end of data to the child */
    waitpid(pid, NULL, 0);
    printf("reader: done\n");
    return 0;
}
```

```
$ gcc -std=c11 -Wall -Wextra -pthread -o q4_pipe q4_pipe.c

$ ./q4_pipe
analyser: 6 readings, average 35.5, max 41
reader: done
```

Figure 24: Actual output of q4_pipe

The output matches the expected output exactly: six readings, average (31 + 33 + 36 + 34 + 41 + 38) / 6 = 213 / 6 = 35.5, maximum 41, followed by "reader: done".

4.4 Experiment: Removing close(fd[1]) in the Parent

In q4_pipe_noclose.c the line close(fd[1]) of the parent is removed and nothing else is changed. The program never finishes. The evidence below was captured with ps while the program ran, and by running the program under timeout.

```
$ ./q4_pipe_noclose &   # close(fd[1]) removed in the parent

$ ps -o pid,ppid,stat,wchan:16,comm -C q4_pipe_noclose
  PID  PPID STAT WCHAN            COMMAND
20990     1 S    do_wait          q4_pipe_noclose
20991 20990 S    anon_pipe_read   q4_pipe_noclose
$ timeout 3 ./q4_pipe_noclose; echo "exit status: $?"
exit status: 124 (124 = killed by timeout, program never finished)
```

Figure 25: Actual evidence: parent blocked in do_wait, child blocked in anon_pipe_read, timeout status 124

Explanation. The kernel counts how many descriptors refer to the write end of a pipe and reports end of file only when that count reaches zero. After the child has read the six integers, the pipe is empty, but the parent still holds fd[1] open, so the count is 1 and read() cannot return 0; it blocks (state S, wait channel anon_pipe_read). The parent meanwhile is blocked in waitpid() waiting for the child to terminate (wait channel do_wait). Each process waits for the other: a **deadlock**. The command timeout 3 ./q4_pipe_noclose had to kill the program, giving exit status 124. The rule is that every process must close the pipe ends it does not use.

4.5 Implementation 2: Shared Memory with Semaphores

The reader and the analyser share a small ring of four slots in memory created with mmap(MAP_SHARED | MAP_ANONYMOUS), which the child inherits through fork(). Two process-shared POSIX semaphores provide the synchronisation: empty (initially 4) counts free slots and full (initially 0) counts filled slots. The producer does wait(empty), writes, signal(full); the consumer does wait(full), reads, signal(empty). Because there is only one producer and one consumer, no extra mutex is needed. Since there is no pipe end of file, the end of data is signalled by a sentinel value (−1).

Listing 11: q4_shm.c – shared memory and semaphores

```
#define _POSIX_C_SOURCE 200809L
#define _DEFAULT_SOURCE
#include <stdio.h>
#include <unistd.h>
#include <semaphore.h>
```

```
#include <sys/mman.h>
#include <sys/wait.h>
#define SLOTS 4
#define END (-1)                    /* sentinel reading: no more data */

struct shared { int buf[SLOTS]; int in, out; sem_t empty, full; };

int main(void)
{
    struct shared *s = mmap(NULL, sizeof *s, PROT_READ | PROT_WRITE,
    MAP_SHARED | MAP_ANONYMOUS, -1, 0);
    if (s == MAP_FAILED) { perror("mmap"); return 1; }
    sem_init(&s->empty, 1, SLOTS);      /* pshared = 1: usable between processes */
    sem_init(&s->full, 1, 0);
    s->in = s->out = 0;
    pid_t pid = fork();
    if (pid == 0) {                     /* analyser: consumer */
        int count = 0, sum = 0, max = 0;
        for (;;) {
            sem_wait(&s->full);
            int x = s->buf[s->out]; s->out = (s->out + 1) % SLOTS;
            sem_post(&s->empty);
            if (x == END) break;
            count++; sum += x; if (x > max) max = x;
        }
        printf("analyser: %d readings, average %.1f, max %d\n", count, count ?
        (double)sum / count : 0.0, max);
        fflush(stdout); _exit(0);
    }
    int readings[] = {31, 33, 36, 34, 41, 38, END};   /* sensor reader: producer */
    for (int i = 0; i < 7; i++) {
        sem_wait(&s->empty);
        s->buf[s->in] = readings[i]; s->in = (s->in + 1) % SLOTS;
        sem_post(&s->full);
```

```
    }
    waitpid(pid, NULL, 0);
    printf("reader: done\n");
    sem_destroy(&s->empty); sem_destroy(&s->full); munmap(s, sizeof *s);
    return 0;
}
```

```
$ gcc -std=c11 -Wall -Wextra -pthread -o q4_shm q4_shm.c

$ ./q4_shm
analyser: 6 readings, average 35.5, max 41
reader: done
```

Figure 26: Actual output of q4_shm

4.6 Growth Investigation: 1 MB, 10 MB and 100 MB

q4_bench.c transfers 1 MB, 10 MB and 100 MB from a producer process to a consumer process in 64 KB chunks. With the pipe, the producer fills a private buffer and calls write(), and the consumer calls read() into its own buffer. With shared memory, a ring of eight 64 KB slots and two semaphores is used, and the producer fills the shared slot directly. In both cases the consumer computes a checksum of every byte, and the program verifies that both methods deliver identical data. Each size is repeated five times and the fastest time is reported.

Listing 12: q4_bench.c – the two transfer methods (extract)

```
/* Pipe: producer fills a private buffer, write() copies it into the kernel, reader's read() copies
it out again */
static double via_pipe(size_t total, unsigned long *chk)
{
    int fd[2]; if (pipe(fd)) exit(1);
    int rp[2]; if (pipe(rp)) exit(1);          /* result pipe: child -> parent */
    double t0 = now(); pid_t pid = fork();
    if (pid == 0) {
        close(fd[1]); close(rp[0]); unsigned char *b = malloc(CHUNK); unsigned long s =
        0; ssize_t n;
        while ((n = read(fd[0], b, CHUNK)) > 0) s += sum_bytes(b, (size_t)n);
        if (write(rp[1], &s, sizeof s) < 0) { _exit(1); }
        _exit(0);
```

```
    }
    close(fd[0]); close(rp[1]); unsigned char *b = malloc(CHUNK);
    for (size_t sent = 0; sent < total; sent += CHUNK) { memset(b, (int)(sent / CHUNK) &
    0xff, CHUNK); if (write(fd[1], b, CHUNK) < 0) exit(1); }
    close(fd[1]); if (read(rp[0], chk, sizeof *chk) < 0) exit(1); waitpid(pid, NULL, 0);
    close(fd[0]); close(rp[0]); free(b); return now() - t0;
}
/* Shared memory ring: no data goes through the kernel; semaphores provide the
synchronisation */
struct ring { unsigned char slot[SLOTS][CHUNK]; sem_t empty, full; unsigned long result;
};
static double via_shm(size_t total, unsigned long *chk)
{
    struct ring *r = mmap(NULL, sizeof *r, PROT_READ|PROT_WRITE,
    MAP_SHARED|MAP_ANONYMOUS, -1, 0);
    sem_init(&r->empty, 1, SLOTS); sem_init(&r->full, 1, 0);
    size_t chunks = total / CHUNK; double t0 = now(); pid_t pid = fork();
    if (pid == 0) { unsigned long s = 0; for (size_t k = 0; k < chunks; k++) { sem_wait(&r-
    >full); s += sum_bytes(r->slot[k % SLOTS], CHUNK); sem_post(&r->empty); } r-
    >result = s; _exit(0); }
    for (size_t k = 0; k < chunks; k++) { sem_wait(&r->empty); memset(r->slot[k %
    SLOTS], (int)k & 0xff, CHUNK); sem_post(&r->full); }
    waitpid(pid, NULL, 0); *chk = r->result; double t = now() - t0; munmap(r, sizeof *r);
    return t;
}
```

```
$ gcc -std=gnu11 -O2 -Wall -Wextra -pthread -o q4_bench q4_bench.c

$ ./q4_bench
Size          Pipe (ms) Shared mem (ms)   Pipe/Shm  checksums equal
1    MB           1.19           0.60       1.98  yes
10   MB           9.97           3.03       3.29  yes
100  MB          86.10          25.55       3.37  yes
```

Figure 27: Actual output of q4_bench

Table 27: Transfer times

Data size	Pipe (ms)	Shared memory (ms)	Pipe / shared memory
1 MB	1.19	0.60	1.98
10 MB	9.97	3.03	3.29
100 MB	86.10	25.55	3.37

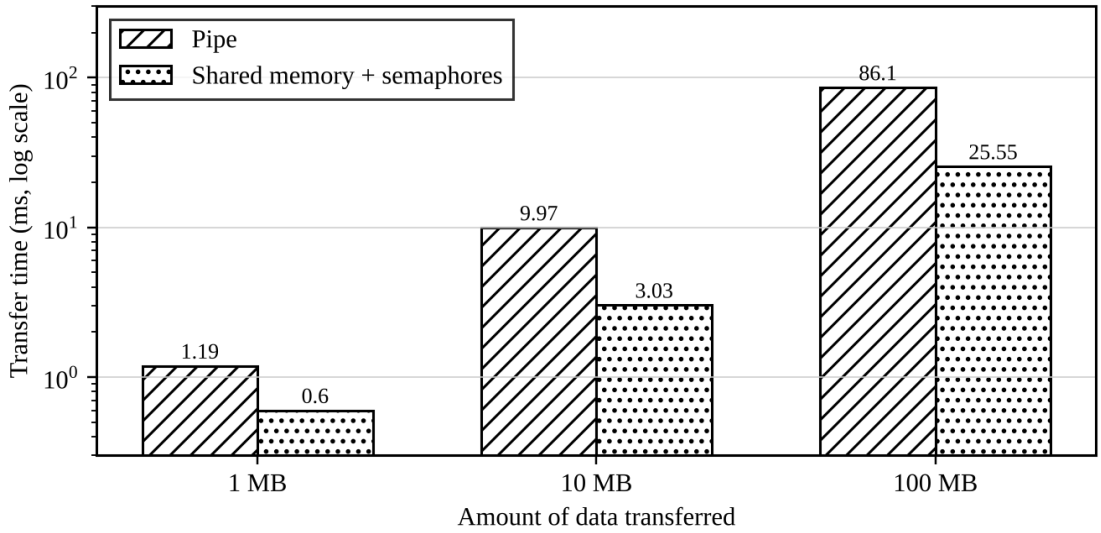


Figure 28: Transfer time against data size (logarithmic scale)

Evaluation. Both methods are linear in the amount of data, O(n): the time per MB is about 1.0 ms (pipe) and 0.3 ms (shared memory) for 10 MB and above (0.86 ms and 0.26 ms for 100 MB). Shared memory is about **3.3 times faster** for large transfers, because it avoids the two kernel copies and a system call for every 64 KB (the pipe buffer is only 64 KB, so the writer and reader also alternate through the scheduler many times). At 1 MB the advantage is only 2×, because a fixed cost (creating the child process, about 0.3–0.5 ms) is a large part of the total. Both methods still pay for producing and checksumming the data in user space, which is why the ratio does not grow without limit.

4.7 Alternative Concept and Complexity Analysis: Shared Memory with a Semaphore Instead of a Pipe

Solution. The readings are written into shared memory by the sensor reader and read in place by the analyser, as in q4_shm.c. Compared with the pipe, this changes the solution in three ways: the end of data is signalled by a sentinel or a counter instead of by closing a descriptor,

the programmer must provide the synchronisation, and the readings are not copied by the kernel.

Table 28: Complexity of transferring n readings

Aspect	Pipe	Shared memory with semaphores
Time to transfer n bytes	O(n): two copies of every byte through the kernel plus about n / 64 KB system calls on each side	O(n) for producing and consuming the data in user space, but with no kernel copies; one semaphore operation per chunk
Measured at 100 MB	86.1 ms	25.6 ms
Space	O(1): kernel buffer of 64 KB	O(1) with a ring buffer (here 8 × 64 KB), or O(n) if the whole data is kept in memory
Synchronisation	Provided by the kernel (blocking read/write)	Written by the programmer: semaphores for full/empty slots, plus a mutex if there are several producers or consumers
Risk of error	Low (forgotten close() causes a hang)	Higher: a missing wait/signal causes lost or duplicated readings or a deadlock

Justified choice of method. A weather station sends only a few small readings per second. For that traffic the transfer cost is negligible (a pipe moves 1 MB in about 1 ms), and the pipe is simpler, automatically synchronised and flow-controlled, and signals end of data by itself. The **pipe is therefore the right choice for the weather-station pipeline**. Shared memory with semaphores becomes the better choice if the station produces bulk data, such as images or thousands of readings per second, where the measured 3-fold advantage matters and the extra synchronisation code is justified.

Question 5 – Synchronisation: Critical Sections, Semaphores and Monitors

5.1 Problem Understanding and Design Starting Point

Thousands of agents book seats on the same trains at the same time. Without synchronisation the same seat is sometimes sold twice; with a single global lock the system becomes very slow. The design starting point is the principle: **a solution to the critical-section problem must guarantee mutual exclusion, progress and bounded waiting.** This principle is converted into a design step by step, from Peterson's two-process solution, through semaphores and the classical problems, to monitors, and a locking granularity for the booking system is finally chosen by measurement.

Table 29: The three requirements of a critical-section solution

Requirement	Meaning	Booking-system example
Mutual exclusion	At most one process is inside the critical section at any time	Two agents never update the same seat counter simultaneously
Progress	If no process is in the critical section, one of the processes that want to enter must be able to enter without being blocked by a process that is not interested	An idle agent cannot prevent others from booking
Bounded waiting	There is a limit on how many times others may enter before a waiting process gets its turn	No agent waits forever while others keep booking

5.2 Peterson's Solution for Two Processes

Peterson's solution uses two shared variables: flag[2], where flag[i] = true means "process i wants to enter", and turn, which says whose turn it is to yield. For process i (with j = 1 − i):

Listing 13: Peterson's algorithm (process i)

```
do {
    flag[i] = true;          /* I want to enter */
    turn = j;                /* but you may go first */
```

```
    while (flag[j] && turn == j)
        ;                   /* busy wait */
    /* ---- critical section ---- */
    flag[i] = false;        /* exit section */
    /* ---- remainder section ---- */
} while (true);
```

Table 30: Why Peterson's solution satisfies the three requirements

Requirement	Argument
Mutual exclusion	Suppose both processes were inside the critical section. Then both flags are true, so each process passed the test only because turn was equal to its own number. The process that wrote turn last set it to the other process's number, and no later write changes it, so only the other process could have passed. This is a contradiction.
Progress	If process j is not interested, flag[j] = false and process i leaves the loop at once. If both are interested, turn decides, and exactly one of them passes.
Bounded waiting	After process j leaves the critical section it sets flag[j] = false. If it wants to enter again it must set turn = i first, so process i, which is waiting, enters before j can enter a second time: i waits at most one turn of j.

Why it may fail on modern processors. The proof assumes that the memory operations happen in program order. Modern CPUs and compilers reorder instructions to improve speed. In particular, a CPU with a store buffer may execute the load of flag[j] before its own earlier store flag[i] = true (and turn = j) becomes visible to the other core (store–load reordering, allowed even on x86). Both processes can then see the other's flag as false and enter the critical section together. A compiler may also reorder or optimise away the loop reads. The solution works only with sequentially consistent memory: a memory barrier between the stores and the loads, or sequentially consistent atomic variables (as in the C11 version below), is required. In addition the busy waiting wastes CPU time, and the solution is defined only for two processes. For these reasons real systems use hardware-supported locks (atomic instructions) and blocking primitives instead.

Listing 14: q5_peterson.c – C11 atomics (default) or plain volatile variables (-DPLAIN)

```
#define N 20000
#ifdef PLAIN
static volatile int flag[2], turn;
#define FLAG_SET(i,v) (flag[i] = (v))
#define FLAG_GET(i)   (flag[i])
#define TURN_SET(v)   (turn = (v))
#define TURN_GET()    (turn)
#else
static atomic_int flag[2], turn;
#define FLAG_SET(i,v) atomic_store(&flag[i], (v))
#define FLAG_GET(i)   atomic_load(&flag[i])
#define TURN_SET(v)   atomic_store(&turn, (v))
#define TURN_GET()    atomic_load(&turn)
#endif
static long counter = 0;
static void *proc(void *arg)
{
    int i = (int)(long)arg, j = 1 - i;
    for (int k = 0; k < N; k++) {
        FLAG_SET(i, 1); TURN_SET(j);              /* entry section */
        while (FLAG_GET(j) && TURN_GET() == j) ;    /* busy wait     */
        counter++;                                /* critical section */
        FLAG_SET(i, 0);                           /* exit section  */
    }
    return NULL;
}
```

```
$ ./q5_peterson          # C11 seq_cst atomics
Peterson (C11 seq_cst atomics): counter = 40000, expected 40000
real    0m0.005s
$ ./q5_peterson_plain    # plain volatile, no fences (-DPLAIN)
Peterson (plain volatile, no fences): counter = 40000, expected 40000
real    0m0.003s
```

Figure 29: Actual output of Peterson's solution (atomic and plain versions, 20,000 iterations per thread)

Both versions produced the correct value 40,000. For the atomic version this is guaranteed. For the plain version the correct result on this machine is not a proof of correctness: the failure needs two cores running at the same time, and the test machine has a single CPU core, on which a thread always sees its own stores. The failure of the plain version is a known property of the hardware that could not be reproduced here.

5.3 Semaphores Protecting the Seat Counter

A semaphore is an integer variable accessed only through two atomic operations: **wait(S)** (also P): if S > 0 then S = S − 1, otherwise the caller blocks; and **signal(S)** (also V): S = S + 1 and one blocked process, if any, is woken. A **binary semaphore** has the value 0 or 1 and works as a lock; a **counting semaphore** can hold any non-negative value and counts the units of a resource. They protect the seat counter in two ways:

- **Binary semaphore** (initial value 1): wait(mutex); if (seats_left > 0) seats_left--; signal(mutex); the test and the decrement become one indivisible step.
- **Counting semaphore** (initial value = number of free seats): every free seat is a token. An agent does a non-blocking wait (sem_trywait): if it succeeds a seat was sold, otherwise the train is full. No separate counter is needed.

q5_seats.c has 8 agents making 20 booking attempts each (160 requests) for 100 seats. In the unprotected version a sched_yield() between reading and writing the counter forces the interleaving that happens by chance on a real multi-core machine.

Listing 15: q5_seats.c – agent thread with the three modes

```
static void *agent(void *arg)
{
    long id = (long)arg;
    for (int i = 0; i < TRIES; i++) {
        if (mode == 0) {                    /* check-then-act without protection */
```

```
            if (seats_left > 0) { int t = seats_left; sched_yield(); seats_left = t - 1;
            sold[id]++; }
        } else if (mode == 1) {                 /* wait(bin) ... signal(bin) */
            sem_wait(&bin);
            if (seats_left > 0) { int t = seats_left; sched_yield(); seats_left = t - 1;
            sold[id]++; }
            sem_post(&bin);
        } else {                                /* each free seat is one token of the counting
        semaphore */
            if (sem_trywait(&seats) == 0) sold[id]++;
        }
    }
    return NULL;
}
```

```
$ gcc -std=gnu11 -O2 -Wall -Wextra -pthread -o q5_seats q5_seats.c

$ ./q5_seats
No protection                                          seats sold = 160 of 100  -> OVERSOLD
Binary semaphore (wait/signal around check+decrement)  seats sold = 100 of 100  -> correct
Counting semaphore initialised to 100                  seats sold = 100 of 100  -> correct
```

Figure 30: Actual output of q5_seats

Without protection **160 seats were sold for 100 seats**: every request succeeded because all agents saw a positive counter before any of them decremented it, which is the double-selling problem of the case study. With either semaphore exactly 100 seats are sold.

5.4 Classical Synchronisation Problems with Semaphores

Bounded buffer (producer–consumer)

A producer puts items into a buffer of SIZE slots and a consumer takes them out. Three semaphores are used: **empty** (initially SIZE) counts free slots, **full** (initially 0) counts filled slots and **mutex** (initially 1) protects the buffer indices. The order of the waits is important: waiting for empty or full **before** taking the mutex prevents a deadlock in which a process sleeps while holding the mutex.

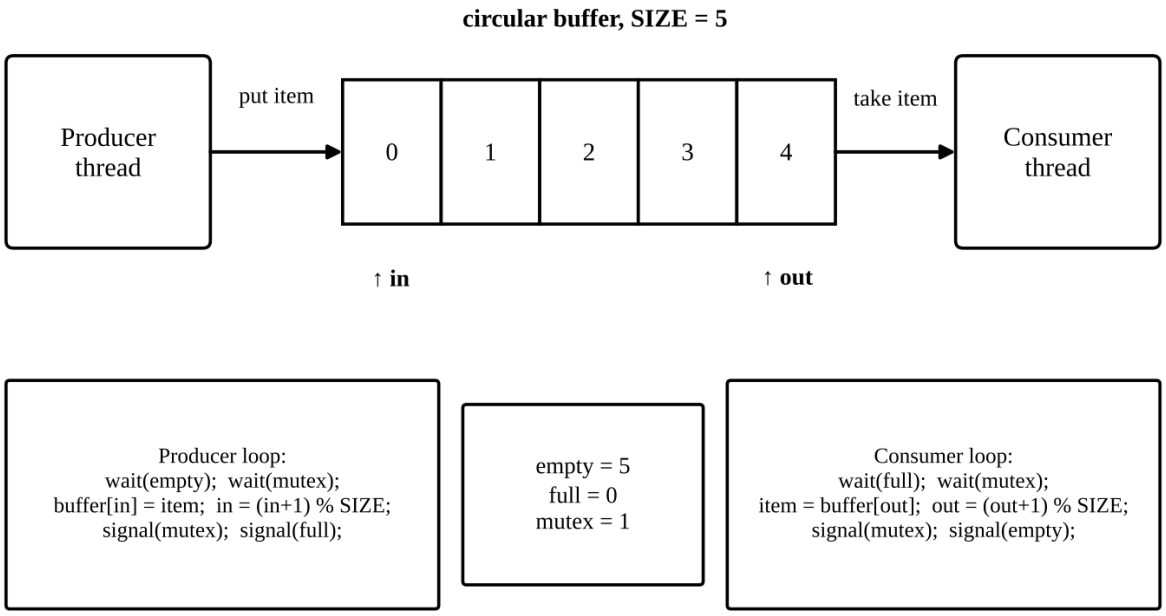


Figure 31: Bounded buffer with the semaphores empty, full and mutex

Readers–writers

Several readers may read a shared object at the same time, but a writer needs exclusive access. The solution (readers have priority) uses a semaphore rw_mutex = 1 for writers and for the first/last reader, a semaphore mutex = 1 protecting read_count, and an integer read_count. The first reader locks rw_mutex, so writers are kept out while any reader is active; the last reader releases it. The writer simply does wait(rw_mutex) … signal(rw_mutex). This solution can starve writers if readers keep arriving; a writer-priority variant avoids this and may starve readers.

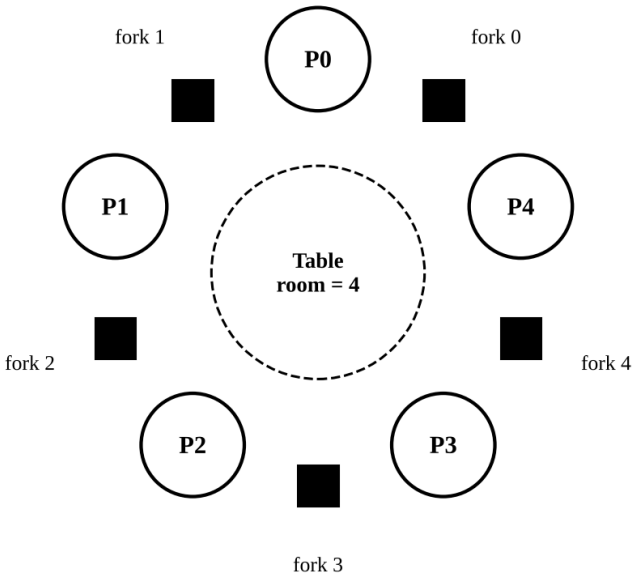
Listing 16: Readers–writers part of q5_rw.c

```
/* ---------- readers-writers ---------- */
static sem_t rw_mutex, mutex_rc; static int read_count = 0, A = 0, B = 0, bad_reads = 0,
reads_done = 0;
static void *writer(void *a){ (void)a; for (int i = 0; i < 2000; i++) { sem_wait(&rw_mutex);
A++; sched_yield(); B++; sem_post(&rw_mutex); } return NULL; }
static void *reader(void *a)
{
    (void)a;
    for (int i = 0; i < 2000; i++) {
```

```
        sem_wait(&mutex_rc); if (++read_count == 1) sem_wait(&rw_mutex);
        sem_post(&mutex_rc);
        if (A != B) bad_reads++;                    /* a reader must never see A != B */
        sched_yield();
        sem_wait(&mutex_rc); reads_done++; if (--read_count == 0)
        sem_post(&rw_mutex); sem_post(&mutex_rc);
    }
    return NULL;
}
```

Dining philosophers

Five philosophers share five forks (one semaphore each, initial value 1). Philosopher i needs forks i and (i + 1) mod 5. If every philosopher picks up the left fork first, all five may hold one fork and wait forever (deadlock). The solution below uses an additional counting semaphore **room** initialised to 4: at most four philosophers may try to eat at the same time, so at least one philosopher can always obtain both forks, which breaks the circular wait.



■ = fork (semaphore initialised to 1); room semaphore initialised to 4
Philosopher i picks up fork i (left) and fork (i+1) mod 5 (right)

Figure 32: Dining philosophers: forks, philosophers and the room semaphore

Listing 17: Dining-philosophers part of q5_rw.c

```
/* ---------- dining philosophers: room semaphore lets at most 4 sit, so a cycle cannot form --
-------- */
#define PHIL 5
static sem_t fork_s[PHIL], room; static int meals[PHIL];
static void *philosopher(void *a)
{
    int i = (int)(long)a;
    for (int m = 0; m < 3; m++) {
        sem_wait(&room);
        sem_wait(&fork_s[i]); sem_wait(&fork_s[(i + 1) % PHIL]);
        meals[i]++; usleep(1000);                    /* eat */
        sem_post(&fork_s[(i + 1) % PHIL]); sem_post(&fork_s[i]);
        sem_post(&room);
        usleep(500);                                 /* think */
    }
    return NULL;
}
```

```
$ ./q5_monitor 1 ; ./q5_monitor 4
Monitor bounded buffer, 1 producer(s)/1 consumer(s): 5000 of 5000 items consumed exactly once -> CORRECT
Monitor bounded buffer, 4 producer(s)/4 consumer(s): 20000 of 20000 items consumed exactly once ->
      CORRECT
$ ./q5_rw
Readers-writers: 3 readers x 2000 reads, 2 writers x 2000 writes -> reads done = 6000, inconsistent reads
       = 0, final A = 4000, B = 4000
Dining philosophers: meals eaten = 3 3 3 3 3  (each philosopher finished, no deadlock)
```

Figure 33: Actual output of the monitor bounded buffer and of q5_rw (readers–writers and dining philosophers)

In the readers–writers test, three readers each read 2,000 times and two writers each write 2,000 times; a writer changes two variables A and B in two steps and a reader checks that A equals B. The result is 6,000 completed reads and **0 inconsistent reads**, and A = B = 4,000. In the philosophers test every philosopher ate 3 times and the program terminated, so there was no deadlock.

5.5 Monitors with Condition Variables

A **monitor** is a language-level construct that bundles shared data with the procedures that operate on it. The monitor guarantees that only one thread is active inside at a time, so the

programmer does not write the locking around every operation. Threads that cannot continue wait on **condition variables** inside the monitor: wait(cond) releases the monitor lock and suspends the thread, and signal(cond) wakes one waiting thread. C has no monitor keyword, so a monitor is built with one pthread mutex and condition variables.

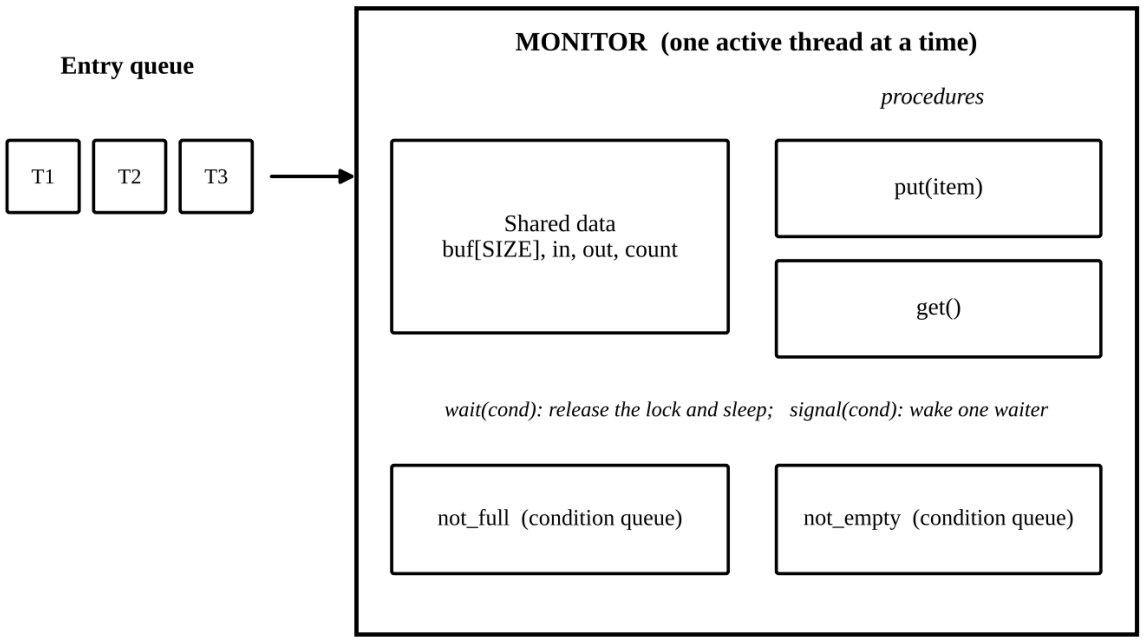


Figure 34: Monitor for the bounded buffer: entry queue, shared data, procedures and condition queues

Listing 18: q5_monitor.c – bounded buffer as a monitor (procedures put and get)

```
static void put(int item)                    /* monitor procedure */
{
    pthread_mutex_lock(&mon.m);
    while (mon.count == SIZE) pthread_cond_wait(&mon.not_full, &mon.m);
    mon.buf[mon.in] = item; mon.in = (mon.in + 1) % SIZE; mon.count++;
    pthread_cond_signal(&mon.not_empty);
    pthread_mutex_unlock(&mon.m);
}
static int get(void)                         /* monitor procedure */
{
    pthread_mutex_lock(&mon.m);
    while (mon.count == 0) pthread_cond_wait(&mon.not_empty, &mon.m);
```

```
    int item = mon.buf[mon.out]; mon.out = (mon.out + 1) % SIZE; mon.count--;
    pthread_cond_signal(&mon.not_full);
    pthread_mutex_unlock(&mon.m);
    return item;
}
```

Table 31: Why a monitor is easier to get right than raw semaphores

Semaphores	Monitor with condition variables
The programmer must write wait/signal pairs around every access; one forgotten signal causes a deadlock and one missing wait causes a race	Mutual exclusion is automatic for every procedure; the shared data cannot be touched from outside
The order of wait(empty) and wait(mutex) is critical; the wrong order deadlocks	No ordering problem: the condition is tested inside the monitor in a while loop, which also copes with spurious wake-ups
The conditions ("buffer not full") are hidden in the counter values of semaphores	The conditions are explicit (count == SIZE, count == 0) and named by condition variables
signal is remembered by the semaphore even if no one is waiting	signal on a condition variable is lost if no one is waiting, so the condition must be tested in a loop (this is part of the monitor discipline)

5.6 Test of the Design: Bounded Buffer ("Complete the Code")

Completed TODOs. TODO 1 (producer): wait on empty, lock the mutex, put the item at buffer[in], advance in circularly, unlock, post full. TODO 2 (consumer): wait on full, lock the mutex, take the item at buffer[out], advance out circularly, unlock, post empty and add the item to consumed_sum. The program is compiled with -pthread.

Listing 19: q5_bb.c – bounded buffer with TODO 1 and TODO 2 completed

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
```

```
#include <pthread.h>
#include <semaphore.h>

#define SIZE 5
#define ITEMS 20

int buffer[SIZE], in = 0, out = 0;
sem_t empty, full;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
long consumed_sum = 0;

void *producer(void *arg)
{
    (void) arg;
    for (int item = 1; item <= ITEMS; item++) {
        /* TODO 1 (completed) */
        sem_wait(&empty);           /* wait for a free slot          */
        pthread_mutex_lock(&lock);
        buffer[in] = item;
        in = (in + 1) % SIZE;
        pthread_mutex_unlock(&lock);
        sem_post(&full);            /* announce one more full slot   */
    }
    return NULL;
}

void *consumer(void *arg)
{
    (void) arg;
    for (int k = 0; k < ITEMS; k++) {
        /* TODO 2 (completed) */
        sem_wait(&full);            /* wait for an item              */
        pthread_mutex_lock(&lock);
        int item = buffer[out];
```

```
        out = (out + 1) % SIZE;
        pthread_mutex_unlock(&lock);
        sem_post(&empty);               /* announce one more free slot     */
        consumed_sum += item;
    }
    return NULL;
}

int main(void)
{
    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
    pthread_t p, c;
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    printf("items consumed: %d, sum = %ld (expected %d)\n", ITEMS, consumed_sum,
           ITEMS * (ITEMS + 1) / 2);
    sem_destroy(&empty);
    sem_destroy(&full);
    return 0;
}
```

```
$ gcc -std=gnu11 -O2 -Wall -Wextra -pthread -o q5_bb q5_bb.c -pthread

$ ./q5_bb
items consumed: 20, sum = 210 (expected 210)
$ for i in $(seq 100); do ./q5_bb; done | sort | uniq -c
    100 items consumed: 20, sum = 210 (expected 210)
```

Figure 35: Actual output of q5_bb: one run, and 100 consecutive runs counted with sort | uniq -c

The single run prints "items consumed: 20, sum = 210 (expected 210)", as expected. In the 100 consecutive runs all 100 printed identical lines with sum = 210, so the result is reproducible.

Several producers and consumers: every item consumed exactly once

q5_bb_multi.c generalises the buffer to N producers and N consumers (5,000 items per producer, buffer of 5 slots). Each item carries a unique number, and every consumption increments an atomic counter for that number, so after the run the program can count how many items were consumed exactly once, never, or more than once.

Listing 20: q5_bb_multi.c – producer and consumer with optional -DNO_MUTEX and -DINJECT

```
static void *producer(void *arg)
{
    long id = (long)arg;
    for (int k = 0; k < ITEMS; k++) {
        sem_wait(&empty); LOCK();
        int idx = in; MAYBE_YIELD();
        buffer[idx] = (int)(id * ITEMS + k); in = (idx + 1) % SIZE;
        UNLOCK(); sem_post(&full);
    }
    return NULL;
}
static void *consumer(void *arg)
{
    (void)arg;
    for (int k = 0; k < np * ITEMS / nc; k++) {
        sem_wait(&full); LOCK();
        int idx = out; MAYBE_YIELD();
        int item = buffer[idx]; out = (idx + 1) % SIZE;
        UNLOCK(); sem_post(&empty);
        atomic_fetch_add(&seen[item], 1);
    }
    return NULL;
}
```

```
$ ./q5_bb_multi 1
1 producer(s), 1 consumer(s): items produced = 5000, consumed exactly once = 5000, never consumed = 0,
      consumed more than once = 0  -> CORRECT
$ ./q5_bb_multi 4
4 producer(s), 4 consumer(s): items produced = 20000, consumed exactly once = 20000, never consumed = 0,
      consumed more than once = 0  -> CORRECT
$ ./q5_bb_nomutex 4   # mutex removed (-DNO_MUTEX), 3 runs
4 producer(s), 4 consumer(s): items produced = 20000, consumed exactly once = 20000, never consumed = 0,
      consumed more than once = 0  -> CORRECT
4 producer(s), 4 consumer(s): items produced = 20000, consumed exactly once = 20000, never consumed = 0,
      consumed more than once = 0  -> CORRECT
4 producer(s), 4 consumer(s): items produced = 20000, consumed exactly once = 20000, never consumed = 0,
      consumed more than once = 0  -> CORRECT
$ ./q5_bb_nomutex_inject 4   # mutex removed + forced context switch (-DINJECT), 3 runs
4 producer(s), 4 consumer(s): items produced = 20000, consumed exactly once = 264, never consumed =
      13669, consumed more than once = 6067  -> WRONG
4 producer(s), 4 consumer(s): items produced = 20000, consumed exactly once = 2580, never consumed =
      11033, consumed more than once = 6387  -> WRONG
4 producer(s), 4 consumer(s): items produced = 20000, consumed exactly once = 6, never consumed = 13964,
      consumed more than once = 6030  -> WRONG
```

Figure 36: Actual output with 1 and 4 producers/consumers, and with the mutex removed

With **1** producer and consumer all 5,000 items were consumed exactly once. With **4** producers and consumers all 20,000 items were consumed exactly once, so the semaphores plus the mutex give correct results.

Removing the mutex. The semaphores still count slots correctly, but several producers can now read the same value of in, write into the same slot and increment in twice (or several consumers read the same out), so items are overwritten or read twice. In three runs with natural scheduling on this single-core machine the result was still correct, because the threads were almost never switched inside the few instructions that matter, so correctness here is by luck and not by design. With a context switch forced inside the unprotected region (-DINJECT) the result was wrong in all three runs: of 20,000 items only between 6 and 2,580 were consumed exactly once, 11,033 to 13,964 were never consumed (overwritten) and 6,030 to 6,387 were consumed twice or more. The mutex is therefore required as soon as there is more than one producer or more than one consumer; with exactly one producer and one consumer, each index is touched by only one thread, and the original program would work even without the mutex.

5.7 Deployment Decision: Lock Granularity

Three designs for the booking system are compared: **one global lock** for the whole system, **one lock per train** (8 locks) and **one lock per coach** (80 locks). The benchmark q5_locks.c starts 8 agent threads that each make 1,500 booking requests for random seats of 8 trains with 10 coaches of 72 seats. The program checks that the number of successful bookings equals the number of booked seats, to prove that no seat is sold twice. Two critical sections are measured: (1) only the check and update of the seat (pure CPU), and (2) the update plus a 50 µs write of

the booking record (modelled by nanosleep), as a real system writes to a journal or database before confirming.

Listing 21: q5_locks.c – agent thread and lock selection

```
static pthread_mutex_t *pick(int t, int c){ return granularity == 0 ? &global_lock :
granularity == 1 ? &train_lock[t] : &coach_lock[t][c]; }

static void *agent(void *arg)
{
    long id = (long)arg; unsigned s = 12345u + (unsigned)id * 7919u;
    for (int i = 0; i < BOOKINGS; i++) {
        s = s * 1103515245u + 12345u; int t = (s >> 8) % TRAINS;
        s = s * 1103515245u + 12345u; int c = (s >> 8) % COACHES;
        s = s * 1103515245u + 12345u; int seat = (s >> 8) % SEATS;
        pthread_mutex_t *m = pick(t, c);
        pthread_mutex_lock(m);
        if (!booked[t][c][seat]) {
            if (io_ns) nanosleep(&(struct timespec){0, io_ns}, NULL);      /* e.g. journal /
            database write */
            booked[t][c][seat] = 1; ok_count[id]++;
        }
        pthread_mutex_unlock(m);
    }
    return NULL;
}
```

```
$ gcc -std=gnu11 -O2 -Wall -Wextra -pthread -o q5_locks q5_locks.c

$ ./q5_locks
8 agent threads x 1500 booking requests, 8 trains x 10 coaches x 72 seats, CPUs online: 1

Critical section = check + update only (CPU only)
Locking scheme              Time (ms)     Bookings/s    Booked double-booking?
One global lock                   0.7       17533837      5055 none
One lock per train (8)            0.7       18356062      5055 none
One lock per coach (80)           0.6       19205531      5055 none

Critical section = update + 50 us booking-record write (I/O wait)
Locking scheme              Time (ms)     Bookings/s    Booked double-booking?
One global lock                 681.0          17621      5055 none
One lock per train (8)          168.8          71087      5055 none
One lock per coach (80)          91.0         131803      5055 none
```

Figure 37: Actual output of q5_locks

Table 32: Measured throughput (8 threads, 12,000 requests, 5,055 successful bookings, no double booking)

Locking scheme	CPU-only critical section: time (ms)	With 50 µs record write: time (ms)	With 50 µs record write: bookings per second
One global lock	0.7	681.0	17,621
One lock per train (8)	0.7	168.8	71,087
One lock per coach (80)	0.6	91.0	131,803

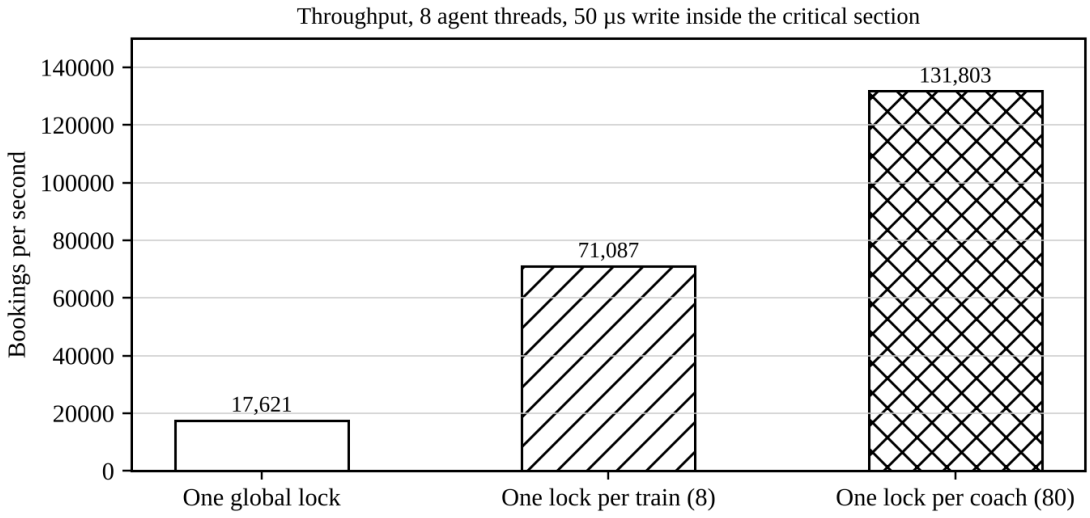


Figure 38: Throughput of the three locking granularities with a 50 µs write in the critical section

Evaluation. When the critical section takes only a few nanoseconds (the seat check and update), the three schemes are indistinguishable (0.6–0.7 ms in total), because the lock is held only very briefly and the test machine has a single core, so nothing can run in parallel anyway. When the critical section includes a realistic 50 µs write, the global lock serialises all agents: throughput is only 17,621 bookings per second. Per-train locks allow different trains to be booked at the same time (4.0 times faster), and per-coach locks allow almost all agents to proceed in parallel (7.5 times faster than the global lock and 1.9 times faster than per-train locks). No scheme produced a double booking.

Decision: one lock per coach. It gives the highest measured throughput, and conflicts happen only when two agents choose the same coach of the same train at the same time. Its costs are small: 80 mutexes (a few bytes each) instead of one. Two cautions apply: a booking that spans several coaches (for example a family seated in two coaches) must acquire the locks in a fixed order, for example by coach number, to avoid a deadlock, and an operation that needs the state of a whole train (an occupancy report) must take all coach locks of that train or work from a snapshot. If most bookings span coaches, the per-train lock is the simpler choice. A single global lock is rejected because it is the case that the problem statement identified as too slow, and this measurement quantified the loss as 7.5-fold. The results come from a one-core machine; on a multi-core server the CPU-only case would also begin to separate, in favour of the finer locks.

5.8 Alternative Concept and Complexity Analysis: Atomic Operations or a Spin Lock

Solution. If the shared counter is only one integer, the semaphore and the mutex can be replaced by the C11 atomic operation atomic_fetch_add(&counter, 1), which the hardware executes as one indivisible instruction, with no lock at all. Alternatively, a **spin lock** (pthread_spin_lock) protects the update: a thread that cannot get the lock keeps testing it in a loop instead of sleeping. q5_atomic.c performs 1,600,000 updates in total, divided among 1, 4 and 16 threads, with a mutex, a spin lock and an atomic add, and reports the time per update and the CPU time consumed.

Listing 22: q5_atomic.c – worker with the three methods

```
static void *worker(void *a)
{
    (void)a;
    for (long i = 0; i < TOTAL / nthreads; i++) {
```

```
        if (method == 0) { pthread_mutex_lock(&m); plain_counter++;
        pthread_mutex_unlock(&m); }
        else if (method == 1) { pthread_spin_lock(&sp); plain_counter++;
        pthread_spin_unlock(&sp); }
        else atomic_fetch_add(&atomic_counter, 1);
    }
    return NULL;
}
```

```
$ gcc -std=gnu11 -O2 -Wall -Wextra -pthread -o q5_atomic q5_atomic.c

$ ./q5_atomic
1600000 updates in total, CPUs online: 1
Method              Threads    ns/update      Wall (ms)     CPU (ms)
Mutex                     1         24.0           38.4         36.0
Mutex                     4         23.0           36.8         35.9
Mutex                    16         21.2           33.9         33.7
Spin lock                 1          8.3           13.4         13.1
Spin lock                 4         30.1           48.2         44.5
Spin lock                16         76.6          122.6        117.3
Atomic fetch_add          1          6.8           10.8         10.8
Atomic fetch_add          4          7.3           11.7         11.7
Atomic fetch_add         16          8.1           13.0         12.8
```

Figure 39: Actual output of q5_atomic

Table 33: Time per update and CPU consumed

Method	Threads	ns per update	Wall time (ms)	CPU time (ms)
Mutex	1	24.0	38.4	36.0
Mutex	4	23.0	36.8	35.9
Mutex	16	21.2	33.9	33.7
Spin lock	1	8.3	13.4	13.1
Spin lock	4	30.1	48.2	44.5
Spin lock	16	76.6	122.6	117.3
Atomic fetch_add	1	6.8	10.8	10.8
Atomic fetch_add	4	7.3	11.7	11.7
Atomic fetch_add	16	8.1	13.0	12.8

Evaluation. The atomic add is the fastest in every case (6.8–8.1 ns per update) and its cost hardly changes with the number of threads. The mutex costs about 21–24 ns per update and also stays flat. The spin lock is fast for one thread (8.3 ns) but becomes **much slower as threads are added: 30.1 ns with 4 threads and 76.6 ns with 16 threads**, and nearly all the extra time is wasted CPU time (CPU 117 ms for 16 threads against 13 ms for 1 thread). The reason is that on this single core, a thread that is preempted while it holds the lock leaves the other threads spinning until their time slice ends, which accomplishes nothing. On a multi-core machine spinning is cheaper, but it still burns CPU while waiting.

Table 34: Choice of method and complexity

Method	Time per uncontended update	Behaviour when contended	Suitable for
Atomic fetch_add	O(1), a single hardware instruction	Hardware serialises the accesses; cheap and wait-free in practice	One-word counters and flags
Spin lock	O(1)	Wastes CPU in proportion to the wait; very poor if the holder is preempted	Very short critical sections on multi-core machines, when the expected wait is less than a context switch
Mutex / semaphore	O(1) (slightly higher constant)	Waiting threads sleep and use no CPU	Long critical sections (for example the booking record write of 50 µs), or when the holder can block

Which method suits a long critical section? The blocking mutex (or semaphore) is the right choice: the waiting threads sleep and release the CPU instead of wasting it, and the sleep and wake-up cost (a few microseconds) is small compared with the long critical section. A spin lock would waste CPU for the whole duration of the critical section. Atomic operations cannot protect a multi-step critical section at all, such as checking a seat and writing the booking record together. In the booking system, therefore, the seat counter can use atomics, while the booking itself uses the per-coach mutex chosen in Section 5.7.

Question 6 – Deadlock Handling and the Banker's Algorithm

Problem Context

A research centre runs jobs that need several shared resources at once: GPUs, licences for simulation software and high-speed storage. Jobs sometimes wait for each other forever and the whole cluster stops until an administrator intervenes.

A. Characterise Deadlock

A deadlock can occur only if all four necessary (Coffman) conditions hold at the same time. Table 1 shows each condition and how it appears in the cluster.

Table 35: The four necessary conditions for deadlock in the cluster

Condition	Meaning	In the cluster example
Mutual exclusion	At least one resource can be used by only one job at a time	A GPU, a software licence and the storage volume are each held exclusively by one job
Hold and wait	A job holds some resources while waiting for others	J1 holds the GPU and waits for the licence; J2 holds the licence and waits for storage; J3 holds storage and waits for the GPU
No preemption	A resource cannot be forcibly taken from a job	A running simulation cannot lose its GPU or licence without losing its work
Circular wait	A closed chain of jobs, each waiting for a resource held by the next	J1 → J2 → J3 → J1 (see the cycle in the resource-allocation graph below)

The resource-allocation graph (RAG) has job nodes (circles) and resource nodes (rectangles; a dot is one instance). An **assignment edge** R → P (solid) means the job holds the resource; a **request edge** P → R (dashed) means the job is waiting for it. Each resource here has a single instance, so the cycle GPU → J1 → LIC → J2 → STO → J3 → GPU is **both necessary and sufficient** for deadlock: J1, J2 and J3 are deadlocked. J4 holds nothing and waits for nothing, so it is not involved and can still finish.

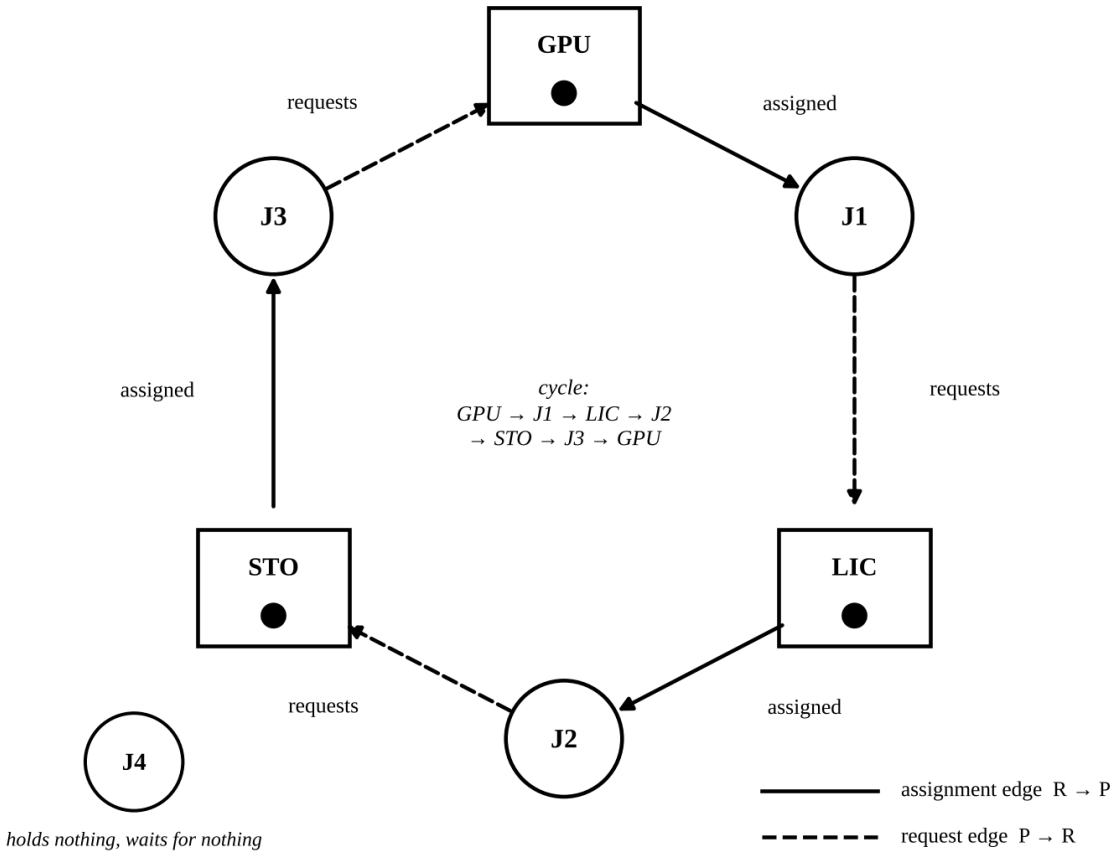


Figure 40: Resource-allocation graph of the cluster with a deadlock among J1, J2 and J3

B. Compare the Strategies

Table 36: Deadlock-handling strategies for the research cluster

Strategy	How it works	Advantages	Disadvantages for the cluster
Prevention (remove one condition)	e.g. a job must request all its resources at once (no hold and wait), or resources are numbered and requested in order (no circular wait)	Deadlock is impossible; no run-time checking; simple to implement in the job scheduler	Lower utilisation: GPUs and licences sit idle while a job holds them early; possible starvation of jobs that need many resources; ordering is hard to impose on all users
Avoidance (Banker's algorithm)	Each job declares its maximum demand; every request is granted only if the resulting state is safe	No deadlock, no aborted jobs, and resources are held only when needed (better utilisation than prevention)	Needs maximum demands in advance, a fixed number of jobs and resources; costs $O(m \cdot n^2)$ per request; conservative, so some safe requests wait
Detection and recovery	Grant requests freely; run a detection algorithm periodically; break a deadlock by aborting or rolling back a job	No advance information; best utilisation; cost only when run	Work is lost when a job is aborted (hours of simulation); deadlocked jobs idle until the next detection run; recovery needs checkpoints
Ignore (ostrich)	Assume deadlock never happens; an administrator restarts the system	Zero cost and zero complexity	Not acceptable here: deadlocks are already happening and stop the whole cluster until an administrator intervenes

Evaluation. Ignoring deadlock is already disproved by the problem statement. Prevention is the easiest to build but wastes expensive resources. Detection and recovery wastes computing

work, which is the main cost in a research cluster. Avoidance is the only strategy that neither wastes resources nor loses work, and its two real costs, the need for declared maximum demands and the computing cost of the check, can be examined with measurements (parts D–F).

C. Apply the Banker's Algorithm

The Banker's algorithm uses, for P processes and R resource types, the vector Available, and the matrices Max, Allocation and Need = Max − Allocation. The given state has 5 processes and 3 resource types with a total of (10, 5, 7) instances.

Table 37: Given state

Process	Allocation (A B C)	Max (A B C)	Need = Max − Allocation	Available
P0	0 1 0	7 5 3	7 4 3	3 3 2
P1	2 0 0	3 2 2	1 2 2	
P2	3 0 2	9 0 2	6 0 0	
P3	2 1 1	2 2 2	0 1 1	
P4	0 0 2	4 3 3	4 3 1	

Safety algorithm

1. Set Work = Available and Finish[i] = false for all i.
2. Find a process i with Finish[i] = false and Need[i] ≤ Work. If none exists, go to step 4.
3. Set Work = Work + Allocation[i], Finish[i] = true, append i to the sequence, and return to step 2.
4. If Finish[i] = true for all i the state is **safe**; otherwise it is **unsafe**.

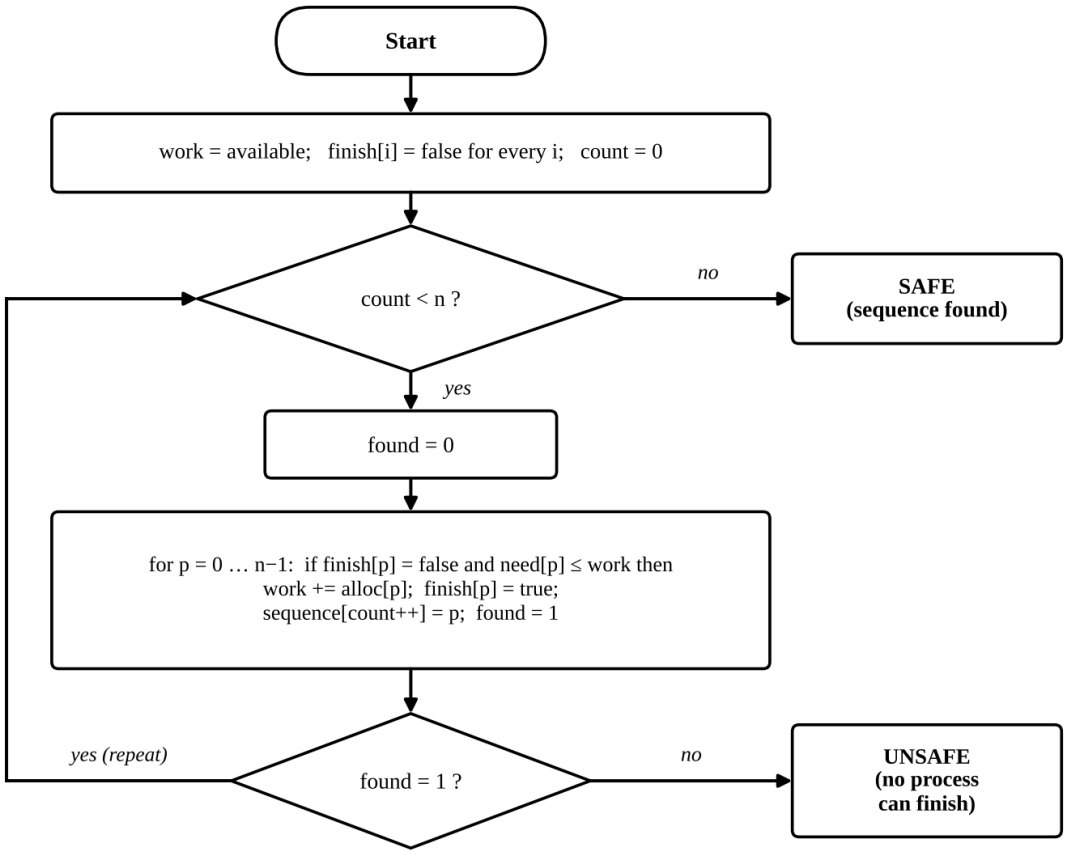


Figure 41: Flowchart of the safety algorithm

Resource-request algorithm

1. If Request[i] > Need[i], raise an error (the job exceeded its maximum claim).
2. If Request[i] > Available, the job must wait.
3. Pretend to allocate: Available −= Request, Allocation[i] += Request, Need[i] −= Request.
4. Run the safety algorithm. If the state is safe, the request is granted; otherwise restore the old state and make the job wait (denied).

Implementation (Complete the Code, TODO 1)

Listing 23: q6_banker.c – safety and request algorithms with TODO 1 completed

```
#include <stdio.h>
#include <string.h>

#define P 5
#define R 3
```

```
int avail[R] = {3, 3, 2};
int maxm[P][R] = {{7, 5, 3}, {3, 2, 2}, {9, 0, 2}, {2, 2, 2}, {4, 3, 3}};
int alloc[P][R] = {{0, 1, 0}, {2, 0, 0}, {3, 0, 2}, {2, 1, 1}, {0, 0, 2}};

/* Returns 1 if the state is safe and stores a safe sequence in seq. */
int is_safe(int seq[P])
{
    int work[R], finish[P] = {0}, count = 0;
    memcpy(work, avail, sizeof work);
    while (count < P) {
        int found = 0;
        for (int p = 0; p < P; p++) {
            if (finish[p]) continue;
            /* TODO 1 (completed): if need = maxm[p] - alloc[p] <= work for every
            resource,
                let p finish, release its allocation, record it in the sequence */
            int can_finish = 1;
            for (int r = 0; r < R; r++)
                if (maxm[p][r] - alloc[p][r] > work[r]) { can_finish = 0; break; }
            if (can_finish) {
                for (int r = 0; r < R; r++) work[r] += alloc[p][r];
                finish[p] = 1;
                seq[count++] = p;
                found = 1;
            }
        }
        if (!found) return 0;
    }
    return 1;
}

void request(int p, const int req[R])
{
```

```
    for (int r = 0; r < R; r++)
        if (req[r] > maxm[p][r] - alloc[p][r] || req[r] > avail[r]) {
            printf("P%d request: must wait (not available)\n", p);
            return;
        }
    for (int r = 0; r < R; r++) { avail[r] -= req[r]; alloc[p][r] += req[r]; }
    int seq[P];
    if (is_safe(seq)) {
        printf("P%d request: granted, safe sequence", p);
        for (int i = 0; i < P; i++) printf(" P%d", seq[i]);
        printf("\n");
    } else {
        for (int r = 0; r < R; r++) { avail[r] += req[r]; alloc[p][r] -= req[r]; }
        printf("P%d request: denied (unsafe)\n", p);
    }
}

int main(void)
{
    int seq[P];
    if (is_safe(seq)) {
        printf("initial state safe:");
        for (int i = 0; i < P; i++) printf(" P%d", seq[i]);
        printf("\n");
    }
    request(1, (int[R]){1, 0, 2});
    request(4, (int[R]){3, 3, 0});
    request(0, (int[R]){0, 2, 0});
    return 0;
}
```

```
$ gcc -std=c11 -Wall -Wextra -o q6_banker q6_banker.c

$ ./q6_banker
initial state safe: P1 P3 P4 P0 P2
P1 request: granted, safe sequence P1 P3 P4 P0 P2
P4 request: must wait (not available)
P0 request: denied (unsafe)
```

Figure 42: Actual output of q6_banker

The output equals the expected output: the initial state is safe with the safe sequence P1 P3 P4 P0 P2; the request of P1 for (1, 0, 2) is granted; the request of P4 for (3, 3, 0) must wait; the request of P0 for (0, 2, 0) is denied as unsafe.

Hand verification of the safe sequence of the initial state

Table 38: Safety algorithm by hand (initial state)

Step	Process chosen	Need ≤ Work?	Work after (Work + Allocation)
0	–	–	(3, 3, 2)
1	P1	(1,2,2) ≤ (3,3,2) yes	(3,3,2) + (2,0,0) = (5,3,2)
2	P3	(0,1,1) ≤ (5,3,2) yes	(5,3,2) + (2,1,1) = (7,4,3)
3	P4	(4,3,1) ≤ (7,4,3) yes	(7,4,3) + (0,0,2) = (7,4,5)
4	P0	(7,4,3) ≤ (7,4,5) yes	(7,4,5) + (0,1,0) = (7,5,5)
5	P2	(6,0,0) ≤ (7,5,5) yes	(7,5,5) + (3,0,2) = (10,5,7)

P0 and P2 were skipped in the first scan because (7,4,3) and (6,0,0) did not fit into Work = (3,3,2) and (5,3,2); they fit later. Work ends at (10, 5, 7), which equals the total number of resource instances, as it must.

Decision on each request

Table 39: Requests in the order of the program

Request	Checks	Decision
P1 requests (1, 0, 2)	Need of P1 = (1,2,2): request ≤ Need. Available (3,3,2): request ≤ Available. New state: Available (2,3,0), Allocation of P1 (3,0,2), Need of P1 (0,2,0). The safety algorithm finds P1, P3, P4, P0, P2.	Granted: safe
P4 requests (3, 3, 0)	Need of P4 = (4,3,1): request ≤ Need. Available is now (2,3,0): the first component 3 > 2.	Must wait: resources not available
P0 requests (0, 2, 0)	Need of P0 = (7,4,3): request ≤ Need. Request ≤ Available (2,3,0). The pretended state is unsafe (below).	Denied: unsafe

Why the request of P0 for (0, 2, 0) leads to an unsafe state (by hand)

The state before the request is the state after P1's request was granted. Available = (2, 3, 0). If P0 were given (0, 2, 0), then Available = (2, 3, 0) − (0, 2, 0) = **(2, 1, 0)**, Allocation of P0 = (0, 3, 0) and Need of P0 = (7, 2, 3).

Table 40: Pretended state after granting (0, 2, 0) to P0

Process	Allocation	Need	Need ≤ Work (2, 1, 0)?
P0	0 3 0	7 2 3	No (7 > 2)
P1	3 0 2	0 2 0	No (2 > 1 in B)
P2	3 0 2	6 0 0	No (6 > 2)
P3	2 1 1	0 1 1	No (1 > 0 in C)
P4	0 0 2	4 3 1	No (4 > 2)

No process has a Need that fits into Work = (2, 1, 0), so no process can be guaranteed to finish: the state is **unsafe**. The system could deadlock if the processes then asked for their maximum

demands, so the Banker's algorithm restores the old state and makes P0 wait, even though 2 units of B are physically available.

D. Predict Before Implementing

Prediction. The safety algorithm has an outer loop that repeats until no process can finish; it can run at most n times because each pass that makes progress finishes at least one process. Each pass examines up to n processes and compares up to m resource types for each. The **worst case is therefore O(m × n²)** (one process finishes per pass, for example when the processes can only finish in the reverse of the scanning order), and the **best case is O(m × n)** (all processes finish in the first pass). The resource-request algorithm adds only O(m) before calling the safety algorithm, so a request costs O(m × n²). **Testable predictions:** (1) when n grows tenfold, the worst-case time grows about 100 times and the best-case time about 10 times; (2) when m doubles, the time doubles.

E. Measure

q6_bench.c times the safety algorithm for n = 10, 100 and 1,000 processes with m = 3 resource types, in two constructed cases: **best case**, in which resources are plentiful and every process finishes in the first pass, and **worst case**, in which process i can finish only after process i + 1, so one process finishes per pass. Each timing repeats the check many times and reports the average.

Listing 24: q6_bench.c – safety check and construction of the two cases (extract)

```
static int safe(int n, int m, const int *avail, const int *need, const int *alloc, int *seq)
{
    int *work = malloc(m * sizeof *work), *finish = calloc(n, sizeof *finish), count = 0;
    memcpy(work, avail, m * sizeof *work);
    while (count < n) {
        int found = 0;
        for (int p = 0; p < n; p++) {
            if (finish[p]) continue;
            int ok = 1; for (int r = 0; r < m; r++) if (need[p*m + r] > work[r]) { ok = 0;
            break; }
            if (ok) { for (int r = 0; r < m; r++) work[r] += alloc[p*m + r]; finish[p] = 1;
            seq[count++] = p; found = 1; }
```

```
            }
            if (!found) { free(work); free(finish); return 0; }
        }
        free(work); free(finish); return 1;
}
/* best = 1: plenty of resources, every process can finish in the first pass.
    best = 0: worst case chain - process i can only finish after process i+1, so one process
    finishes per pass */
static void build(int n, int m, int best, int *avail, int *need, int *alloc)
{
        for (int r = 0; r < m; r++) avail[r] = best ? 1000000 : 1;
        for (int p = 0; p < n; p++) for (int r = 0; r < m; r++) { alloc[p*m + r] = 1; need[p*m + r]
        = (best || r < m - 1) ? 1 : 1 + (n - 1 - p); }
}
```

```
$ gcc -std=gnu11 -O2 -Wall -Wextra -pthread -o q6_bench q6_bench.c

$ ./q6_bench
Safety algorithm, m = 3 resource types
Case                    n   us per check   growth vs n/10
Best (1 pass)          10           0.11                -
Best (1 pass)         100           0.49             x4.4
Best (1 pass)        1000           6.05            x12.3
Worst (n passes)       10           0.20                -
Worst (n passes)      100          12.95            x63.9
Worst (n passes)     1000        1077.55            x83.2

Prediction: best case O(n*m) -> x10 when n grows x10; worst case O(n^2*m) -> x100.

Growth with m (n = 100, worst case):
m      us per check
3             14.37
6             31.48
12            54.46
24           101.19
```

Figure 43: Actual output of q6_bench

Table 41: Measured time of one safety check (m = 3)

Case	n	Time per check (µs)	Growth for n × 10	Predicted growth
Best (1 pass)	10	0.11	–	–
Best (1 pass)	100	0.49	× 4.4	× 10
Best (1 pass)	1000	6.05	× 12.3	× 10
Worst (n passes)	10	0.20	–	–
Worst (n passes)	100	12.95	× 63.9	× 100
Worst (n passes)	1000	1077.55	× 83.2	× 100

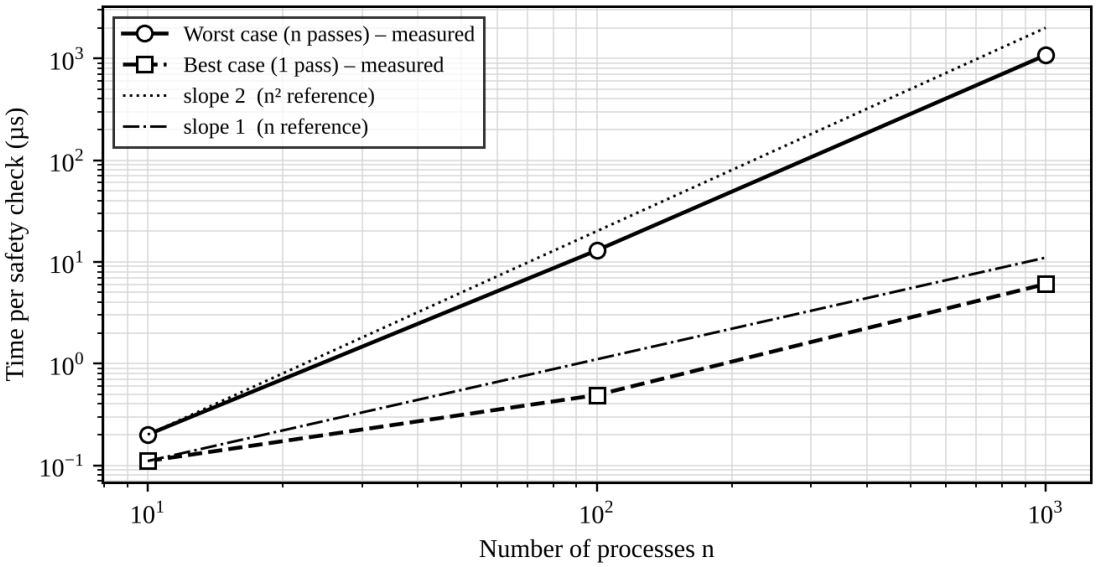


Figure 44: Measured time against number of processes (logarithmic axes)

Table 42: Growth with the number of resource types (n = 100, worst case)

m	Time per check (µs)	Ratio to m = 3
3	14.37	1.0
6	31.48	2.2
12	54.46	3.8
24	101.19	7.0

Comparison with the prediction. The measurements confirm the prediction. In the worst case the time grows by about 64 and 83 times for each tenfold increase of n (close to the predicted 100; the first step is lower because fixed costs such as memory allocation matter at small n), a slope of about 1.8–1.9 on the logarithmic chart, so the algorithm is $O(n^2)$. In the best case the growth is about 10 times (4.4 and 12.3), a slope between 0.6 and 1.1, so it is about O(n). Doubling m roughly doubles the time (3 → 6 → 12 → 24 gives 1.0, 2.2, 3.8, 7.0 times), so the time is linear in m. Hence a check costs $O(m \times n^2)$ in the worst case. In absolute terms, a worst-case check for **1,000 jobs takes about 1 ms**, and for 100 jobs about 13 μs, which is negligible compared with the duration of a job that runs for minutes or hours.

Alternative Concept and Complexity Analysis: Detection and Recovery

Solution. Instead of checking every request, requests are granted whenever resources are free, and a detection algorithm runs periodically (for example every few minutes) on the current Allocation and Request matrices. It is the safety algorithm with Request instead of Need: a job whose current request fits into Work is assumed to finish and release its resources; the jobs that can never finish are deadlocked. Recovery aborts the deadlocked job with the lowest cost (least work lost), releases its resources and runs the detection again. The program below applies this to the cluster of the resource-allocation graph above.

Listing 25: q6_detect.c – detection and recovery (core)

```
/* returns the number of deadlocked jobs; marks them in dead[] */
static int detect(int dead[N])
{
    int work[M], finish[N], changed = 1, k = 0;
    memcpy(work, avail, sizeof work);
    for (int i = 0; i < N; i++) { int z = 1; for (int r = 0; r < M; r++) if (alloc[i][r]) z = 0;
    finish[i] = z; }
    while (changed) {                          /* O(m * n^2) */
        changed = 0;
        for (int i = 0; i < N; i++) {
            if (finish[i]) continue;
            int ok = 1; for (int r = 0; r < M; r++) if (request[i][r] > work[r]) { ok = 0;
            break; }
```

```
            if (ok) { for (int r = 0; r < M; r++) work[r] += alloc[i][r]; finish[i] = 1; changed
            = 1; }
        }
    }
    for (int i = 0; i < N; i++) { dead[i] = !finish[i]; k += dead[i]; }
    return k;
}
int main(void)
{
    int dead[N], k = detect(dead);
    printf("Detection run 1: %d deadlocked job(s):", k);
    for (int i = 0; i < N; i++) if (dead[i]) printf(" J%d", i + 1);
    printf("\n");
    if (k) {
        int victim = -1;
        for (int i = 0; i < N; i++) if (dead[i] && (victim < 0 || lost_minutes[i] <
        lost_minutes[victim])) victim = i;
        printf("Recovery: abort J%d (cheapest victim, %d minutes of work lost) and
        release", victim + 1, lost_minutes[victim]);
        for (int r = 0; r < M; r++) if (alloc[victim][r]) { printf(" %s", rname[r]); avail[r] +=
        alloc[victim][r]; alloc[victim][r] = 0; }
        printf("\n");
        for (int r = 0; r < M; r++) request[victim][r] = 0;
        k = detect(dead);
        printf("Detection run 2: %d deadlocked job(s)%s\n", k, k ? "" : " - the cluster can
        continue");
    }
    return 0;
}
```

```
$ gcc -std=c11 -Wall -Wextra -o q6_detect q6_detect.c

$ ./q6_detect
Detection run 1: 3 deadlocked job(s): J1 J2 J3
Recovery: abort J2 (cheapest victim, 5 minutes of work lost) and release LIC
Detection run 2: 0 deadlocked job(s) - the cluster can continue
```

Figure 45: Actual output of q6_detect

Detection run 1 finds J1, J2 and J3 deadlocked (J4 is not involved). The cheapest victim is J2, which has lost only 5 minutes of work (J1 would lose 40 minutes and J3 90 minutes); aborting J2 releases the licence, J1 can then continue, and detection run 2 finds no deadlock.

Table 43: Banker's algorithm compared with detection and recovery

Aspect	Banker's algorithm (avoidance)	Detection and recovery
Time per request	$O(m \times n^2)$ for every request (about 13 µs for 100 jobs, 1 ms for 1,000 jobs, worst case)	No check at the time of a request: O(1)
Cost of detection	None	$O(m \times n^2)$ per detection run, but only once every few minutes, so negligible
Work lost	None: no job is ever aborted. Some requests wait longer than necessary (conservative)	The work of every aborted job (5 minutes for the cheapest victim here, hours for a long simulation); idle time until the next detection run
Information needed	Maximum demand of every job in advance and a fixed set of jobs	Only current allocations and requests
Resource utilisation	Lower: requests are delayed to stay safe	Highest: every request is granted when possible

F. Defend the Final Design

Recommendation. For the research centre, use **deadlock avoidance with the Banker's algorithm in the job scheduler, with detection and recovery as a safety net.** Jobs declare their maximum GPU, licence and storage demands in the job submission script, the scheduler grants a request only if the state stays safe, and a periodic detection run (every few minutes,

with checkpoint-based abort of the cheapest job) handles the rare case where a job exceeds its declared maximum or a resource fails.

- **Cost of the Banker's algorithm is acceptable.** The measured worst case is about 13 µs for 100 jobs and about 1 ms for 1,000 jobs, per request, while jobs run for minutes or hours. The overhead is far below 0.01 % of a job's run time and does not limit the scheduler.
- **The need for maximum demands in advance is the real limitation, and it is acceptable here.** Cluster batch jobs normally request their resources when they are submitted, so the maxima are known. If users declare too much, resources are reserved but idle; this is the price of avoidance. If a job needs more than it declared, the algorithm rejects the request and the job must be resubmitted, which is the case that detection and recovery covers.
- **Why not prevention alone?** Requesting all resources at once would leave expensive GPUs and licences idle while a job waits for storage. **Why not detection alone?** Each deadlock destroys hours of research computation, whereas avoidance loses none. **Why not ignore?** The cluster is already stopping until an administrator intervenes.
- **Limits of the evidence.** The timings were obtained on a single-core virtual machine with constructed worst and best cases; the real cost depends on the number of jobs and on how the scheduler is implemented. The algorithm also assumes a fixed number of jobs and resource instances; with a very large changing job population it should be applied only to the jobs that are currently running or being admitted, not to all queued jobs.